COLLÈGE DE PARIS SUPÉRIEUR

RÉPUBLIQUE TOGOLAISE

Information Technology

Tronc Commun

Travail – Liberté – Patrie

INF-ASM-B2

Architecture et Programmation en Assembleur

Microcontrôleurs AVR, assembleur x64, bas niveau et systèmes embarqués

Fondements matériels et programmation bas niveau

Architecture des ordinateurs, microcontrôleurs AVR, registres, mémoire, interruptions, timers, E/S, compilation croisée, AVR-GCC, AVRDude, NASM et architecture x64

AVR · Assembleur · Toolchain · TP

Bachelor 2 (IT) | Semestre 2 | 24h (CM + TD + TP) | 2 ECTS

Enseignant : Dr AFENYIVEH Dzo M. Serge

dzo.serge@gmail.com | d.afenyiveh@univkara.com

(+228) 91 75 98 33 | (+228) 98 35 23 31

Collège de Paris Supérieur | 2025–2026

Version 4.1 | Mise à jour : 9 avril 2026

Cours et Travaux Dirigés

Architecture des microcontrôleurs, langage assembleur, programmation bas niveau et mises en pratique

Sommaire

Syllabus du cours **1**

1 Fondements préliminaires **15**
1.1 Systèmes de numération 17
1.2 Représentation des entiers 27
1.3 Logique booléenne 31
1.4 Machine à états et cycle d'exécution 35
1.5 Travaux dirigés du chapitre 0 42
1.6 Quiz du chapitre 0 : testez vos connaissances 53

2 Introduction à l'architecture des microcontrôleurs AVR **59**
2.1 Qu'est-ce qu'un microcontrôleur ? 62
2.2 Petite histoire des microcontrôleurs 68
2.3 Microcontrôleur vs microprocesseur : quelle différence ? 69
2.4 Les microcontrôleurs dans le monde réel 72
2.5 La famille AVR : notre terrain de jeu 74
2.6 Architecture interne : visite guidée de l'ATmega328P 76
2.7 L'horloge, le reset et le cycle d'instruction 84
2.8 Compilation croisée : du PC au µC 88
2.9 Votre premier programme : faire clignoter une LED 91
2.10 Conclusion du chapitre 94

Syllabus du cours

Architecture et Programmation en Assembleur

Comprendre la machine, programmer au plus près du matériel

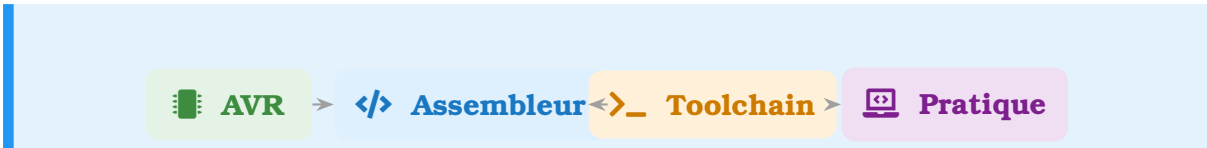
Bachelor 2 (IT) – Tronc commun – Semestre 2

Collège de Paris Supérieur

Description du cours

Ce cours introduit les étudiants à la **programmation bas niveau** et aux fondements de l'**architecture des systèmes numériques**. Il répond à une question centrale : *comment une machine exécute-t-elle réellement un programme ?*

À travers l'étude du langage assembleur et de l'architecture **AVR**, les étudiants découvrent la représentation des données, le fonctionnement du processeur, l'organisation mémoire et l'interaction directe avec le matériel.



Informations générales

Intitulé	Architecture et Programmation en Assembleur
Code	INF-ASM-B2
Niveau	Bachelor 2 (IT)
Semestre	Semestre 2
Volume horaire	**24 heures** (CM : 8h, TD : 8h, TP : 8h)
Crédits	2 ECTS
Institution	Collège de Paris Supérieur
Enseignant	Dr AFENYIVEH Dzo M. Serge
Contact	dzo.serge@gmail.com
Prérequis	• Bases en algorithmique et programmation • Notions de numération binaire et hexadécimale • Autonomie minimale sur ordinateur

Objectifs pédagogiques

À l'issue de ce module, l'étudiant sera capable de :

- **Identifier et expliquer** les composants essentiels d'un microcontrôleur (CPU, ALU, registres, mémoire).
- **Distinguer** langage machine, assembleur et langage de haut niveau.
- **Lire, analyser et interpréter** un programme assembleur AVR.
- **Écrire et déboguer** des programmes assembleur simples avec instructions de transfert, calcul et branchement.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

- **Manipuler** la pile, les sous-programmes et les entrées/sorties numériques.
- **Utiliser** les outils de développement, de simulation et de programmation.

Compétences développées

Conceptuelles
Structure des systèmes numériques

Techniques
Lecture et écriture assembleur

Méthodologiques
Décomposition bas niveau

Pratiques
Simulation et débogage

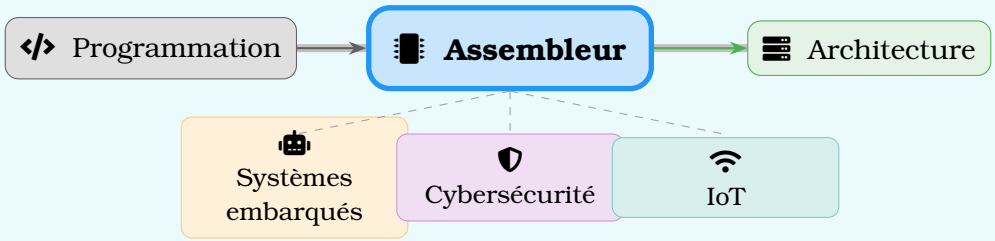
Transversales
Rigueur et analyse

Réf.	Compétence détaillée	Niveau
C1	Comprendre la structure d'un microcontrôleur et le rôle de chaque composant	Maîtrise
C2	Lire, analyser et écrire du code assembleur AVR	Maîtrise
C3	Manipuler registres, mémoire, pile et configurer des E/S	Application
C4	Utiliser les outils de développement et de simulation	Application
C5	Décomposer un problème et raisonner au niveau instruction	Maîtrise

Maîtrise = autonomie attendue **Application** = mise en œuvre guidée

© 2025–2026 Dr AFENYIVEH Dzo M. Serge

Positionnement dans la formation



Ce cours établit un lien fondamental entre la **programmation de haut niveau** et le **fonctionnement matériel**. Il prépare aux enseignements avancés en architecture, systèmes d'exploitation et applications embarquées.

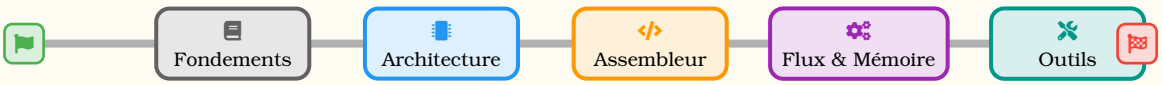
Prérequis

Les prérequis attendus pour ce cours sont :

- ✓ Bases élémentaires en **algorithmique et programmation**
- ✓ Compréhension des **variables, conditions et boucles**
- ✓ Familiarité avec les **nombres binaires et hexadécimaux**
- ✓ Autonomie de base dans l'usage d'un **ordinateur et d'un éditeur**

Une mise à niveau est prévue en début de cours pour harmoniser les acquis.

Programme du cours



 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Chapitre 0 — Fondements préalables 4h

- Numération : binaire, décimale, hexadécimale, conversions
- Représentation des données : entiers signés/non signés, complément à deux
- Logique booléenne : AND, OR, NOT, XOR, tables de vérité
- Notion d'état machine et cycle d'exécution

Chapitre 1 — Architecture des microcontrôleurs AVR 4h

- Introduction aux microcontrôleurs : définition, domaines d'application et présentation de la famille AVR.
- Architecture interne : processeur (CPU), unité arithmétique et logique (ALU) et registres généraux (`R0-R31`).
- Organisation mémoire : mémoire Flash (programme), SRAM (données) et EEPROM (stockage non volatile).
- Horloge système, mécanisme de réinitialisation (*reset*) et cycle d'instruction.
- Comparaison entre microprocesseur et microcontrôleur.

Chapitre 2 — Fondements du langage assembleur 4h

- Du langage machine à l'assembleur : traduction, mnémoniques
- Structure d'un programme : directives, sections, étiquettes, commentaires
- Registres de travail et registres spéciaux (SREG, SP, PC)
- Instructions de transfert : `LDI`, `MOV`, `LDS`, `STS`
- Premier programme : analyse ligne par ligne

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Chapitre 3 — Calculs et contrôle du flux — 4h

- Instructions arithmétiques : `ADD`, `SUB`, `INC`, `DEC`, `MUL`
- Instructions logiques : `AND`, `OR`, `EOR`, `COM`, `NEG`
- Registre d'état SREG : drapeaux Z, C, N, V, S, H
- Comparaisons : `CP`, `CPI`, `TST`
- Branchements : `BREQ`, `BRNE`, `BRLO`, `BRSH`, `RJMP`
- Structures de contrôle : boucles, conditions

Chapitre 4 — Mémoire, pile et sous-programmes — 4h

- Organisation détaillée de la mémoire SRAM
- Modes d'adressage : direct, indirect (X, Y, Z), avec déplacement
- La pile : principe LIFO, pointeur de pile SP, `PUSH`/`POP`
- Sous-programmes : `RCALL`, `RET`, passage de paramètres
- Sauvegarde et restauration du contexte

Chapitre 5 — Entrées/sorties numériques — 4h

- Ports d'E/S : registres DDRx, PORTx, PINx
- Configuration des broches : entrée, sortie, pull-up interne
- Applications pratiques : LED, bouton-poussoir, anti-rebond logiciel
- Temporisations par boucles logicielles
- Introduction aux interruptions et aux timers (principes)

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Chapitre 6 — Outils de développement et ouvertures 2h

- Chaîne de développement : édition → assemblage → simulation → programmation
- Outils : Microchip Studio, AVRDude, Tinkercad Circuits, Proteus
- Débogage : points d'arrêt, pas à pas, inspection des registres
- Ouvertures : systèmes embarqués, IoT, robotique, cybersécurité

Planning des séances

Séance	Chapitre	Contenu principal	Format
S1	Ch. 0	Mise à niveau : numération, logique booléenne, architecture de base	CM + TD
S2	Ch. 1	Architecture AVR : composants internes, mémoire, cycle d'instruction	CM + TD
S3	Ch. 2	Syntaxe assembleur, registres, instructions de transfert	CM + TD
S4	Ch. 3	Calculs arithmétiques/logiques, comparaisons, branchements	TD + TP
S5	Ch. 4–5	Mémoire, pile, sous-programmes, E/S numériques	TP
S6	Ch. 6	Outils, simulation, révision, synthèse et **évaluation finale**	TP + Éval

Volume total : 24 heu **Durée séance :** 4 heure **Période :** 6 semaines

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Approches pédagogiques

- **Cours magistraux** avec démonstrations en direct et illustrations visuelles.
- **Travaux dirigés** : analyse de code, exercices de conversion, traçage d'exécution.
- **Travaux pratiques** : simulation sur Tinkercad/Proteus, manipulation réelle sur Arduino.
- **Progression spiralaire** : les concepts sont revisités avec une complexité croissante.
- **Accompagnement** : exercices corrigés, retours individualisés, sessions de questions.

Principe directeur : Comprendre *pourquoi* avant de coder *comment*. Chaque instruction est reliée à son effet réel sur la machine.

Conseils pour réussir

1. **Assiduité** : assister à toutes les séances (CM, TD, TP).
2. **Révision active** : relire et résumer les notes après chaque cours.
3. **Pratique régulière** : rejouer les exemples sur simulateur.
4. **Traçage manuel** : exécuter les programmes à la main, registre par registre.
5. **Documentation** : commenter systématiquement son code.
6. **Curiosité** : poser des questions sans attendre.
7. **Collaboration** : échanger avec les pairs (sans copier).

© 2025–2026 Dr AFENYIVEH Dzo M. Serge

Modalités d'évaluation

15%
Quiz & Exercices
Vérification régulière

→

35%
TD + TP
Analyse & Pratique

→

50%
Examen final
Synthèse individuelle

Composante	Poids	Description
Quiz et exercices	15%	Vérifications courtes des notions récentes
Travaux dirigés	15%	Analyse, lecture et écriture de séquences assembleur
Travaux pratiques	20%	Simulation, manipulation, tests sur matériel ou simulateur
Examen final	50%	Épreuve individuelle couvrant l'ensemble du programme

Critères d'évaluation

L'évaluation porte sur :

- la **compréhension** des concepts architecturaux ;
- la **rigueur** syntaxique et logique du code ;
- la capacité à **lire et expliquer** du code assembleur ;
- l'aptitude à **produire** une solution correcte et justifiée.

Session de rattrapage : selon le calendrier académique, elle remplace uniquement l'exame

© 2025–2026 Dr AFENYIVEH Dzo M. Serge

Outils et ressources techniques

Environnements de développement

- **Microchip Studio** : IDE complet (édition, assemblage, simulation, débogage)
- **AVRDude** : programmation en ligne de commande
- **Arduino IDE** : support simplifié pour des tests rapides

Simulateurs en ligne

- **Tinkercad Circuits** : simulation gratuite sans installation
- **Wokwi** : simulateur AVR moderne

Simulateurs avancés

- **Proteus** : simulation de circuits complexes
- **SimulIDE** : alternative open source

Matériel pédagogique

- Carte Arduino UNO (ATmega328P)
- LED, résistances (220 Ω, 10 kΩ)
- Boutons-poussoirs
- Plaque d'essai (*breadboard*)
- Câbles de connexion

`Microchip Studio` `Tinkercad` `Proteus` `Arduino`

Supports pédagogiques fournis

- **Polycopié de cours** complet
- **Fiches de synthèse** par chapitre
- **Séries de TD** avec corrigés détaillés
- **Exemples de programmes** commentés
- **Guides de TP** pas à pas
- **Tableaux de référence** (instructions, registres)

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Références bibliographiques

Ouvrages de référence :

[1] **Mazidi, Naimi & Naimi**
AVR Microcontroller and Embedded Systems : Using Assembly and C
Pearson Education. ★★★

[2] **Hyde, R.**
The Art of Assembly Language
No Starch Press. ★★★

[3] **Patterson, D. & Hennessy, J.**
Computer Organization and Design : The Hardware/Software Interface
Morgan Kaufmann. ★★

[4] **Tanenbaum, A. & Austin, T.**
Structured Computer Organization
Pearson. ★★

Documentation technique :

- **AVR Instruction Set Manual** : https://ww1.microchip.com/downloads/en/DeviceDoc/AVR-InstructionSet-Manual-DS40002198.pdf
- **ATmega328P Datasheet** : https://ww1.microchip.com/downloads/en/DeviceDoc/ATmega48A-PA-88A-PA-168A-PA-328-P-DS-DS40002061B.pdf

Ressources en ligne :

- **AVR Freaks** : https://www.avrfreaks.net/ – Communauté et forums
- **Arduino Documentation** : https://docs.arduino.cc/
- **Tinkercad Circuits** : https://www.tinkercad.com/circuits
- **Ben Eater (YouTube)** : https://www.youtube.com/c/BenEater –

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Tutoriels vidéo

⚠ Cadre académique et intégrité

Présence et participation :

- La présence aux CM, TD et TP est **obligatoire**.
- Toute absence doit être justifiée dans les délais prévus par le règlement.
- L'implication active et la régularité du travail sont attendues.

Intégrité académique :
Le plagiat, la copie et toute forme de fraude sont **strictement interdits**. Cela inclut :

- la copie de code sans citation appropriée ;
- l'usage non autorisé d'outils externes pendant les évaluations ;
- toute communication non autorisée pendant une épreuve.

Les infractions sont passibles de sanctions disciplinaires.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Perspectives et prolongements

Architecture
CPU avancés, pipelines

Systèmes
Noyau, pilotes

Embarqué
STM32, ESP32

IoT
Capteurs, actionneurs

Cybersécurité
Reverse engineering

Robotique
Contrôle temps réel

*Ce cours constitue une **fondation essentielle** pour comprendre le fonctionnement interne des systèmes numériques et ouvre des perspectives vers de nombreux domaines de l'informatique et de l'électronique.*

Synthèse du cours

Ce cours repose sur **cinq principes fondamentaux** :

1. **Consolider les bases** avant d'aborder le bas niveau.
2. **Comprendre l'architecture** avant d'écrire du code.
3. **Pratiquer régulièrement** sur des cas simples et progressifs.
4. **Relier chaque instruction** à son effet réel sur la machine.
5. **Ouvrir progressivement** vers les systèmes embarqués et l'IoT.

L'objectif n'est pas seulement d'apprendre une syntaxe, mais d'acquérir une **vision durable** du fonctionnement interne des systèmes numériques.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

> *« Comprendre l'assembleur, c'est voir ce que la machine fait vraiment quand un langage de haut niveau semble simple. »*

Collège de Paris Supérieur

Contact enseignant : dzo.serge@gmail.com

Année académique 2025–2026

Ce syllabus peut faire l'objet d'ajustements mineurs en cours de semestre, sans modification de ses objectifs fondamentaux.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

CHAPITRE I

Fondements préliminaires

Chapitre 0

Avant de programmer la machine, apprenons à penser comme elle

4 heures | Théorie fondamentale | Exercices pratiques

Pourquoi ce chapitre est essentiel

Ce chapitre pose les **fondations scientifiques indispensables** à tout le cours. Avant d'aborder les registres, les instructions assembleur, l'architecture des processeurs ou les systèmes embarqués, il faut comprendre comment une machine numérique :

- **représente les nombres** (binaire, hexadécimal) ;
- **code les valeurs négatives** (complément à deux) ;
- **raisonne en termes logiques** (AND, OR, NOT, XOR) ;
- **change d'état** et **exécute des instructions**.

Ces notions sont omniprésentes dans votre quotidien numérique :

- Calculatrice de poche
- Thermomètre numérique
- Montre digitale
- Feu tricolore automatique
- Télécommande infrarouge
- Smartphone et WiFi

« Une machine numérique ne comprend ni les mots, ni l'intuition humaine : elle comprend des bits, des états et des règles précises. »

◎ Objectifs d'apprentissage

À l'issue de ce chapitre, vous serez capable de :

1. **Convertir** un nombre entre les bases décimale, binaire et hexadécimale.
2. **Expliquer** le principe d'un système de numération positionnel.
3. **Distinguer** une représentation signée d'une représentation non signée.
4. **Appliquer** le complément à deux pour représenter les entiers négatifs.
5. **Construire** et **interpréter** des tables de vérité.
6. **Utiliser** les opérations logiques AND, OR, NOT et XOR.
7. **Décrire** une machine à états finis et le cycle d'exécution d'un processeur.

🔑 Prérequis

Ce chapitre ne suppose pas de connaissances avancées, mais il est utile de :

- ✔ Comprendre les puissances entières ($2^3 = 8$, $10^2 = 100$, etc.)
- ✔ Savoir qu'un signal numérique peut prendre deux états (haut/bas, 0/1)
- ✔ Être capable de suivre un raisonnement logique simple
- ✔ Manipuler des tableaux et des schémas élémentaires

Bonne nouvelle : tout est repris progressivement, avec des exemples concrets !

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

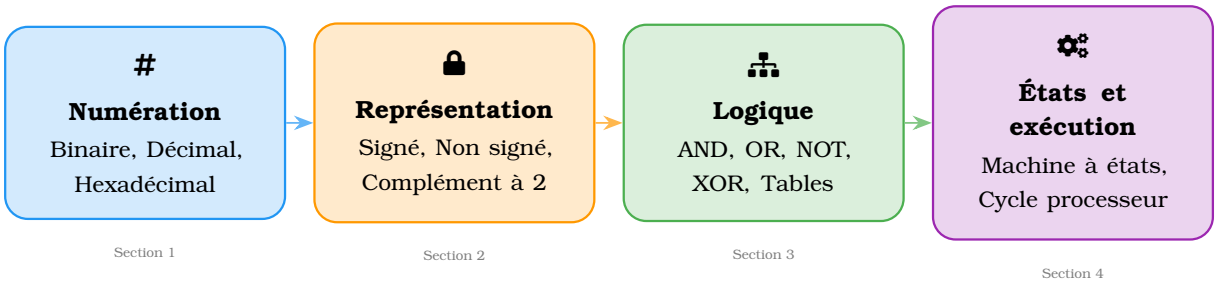


Figure 1.1 – Organisation logique du chapitre : des fondements de la numération jusqu'au cycle d'exécution du processeur.

1.1 Systèmes de numération

1.1.1 Des doigts aux transistors : pourquoi compter en base 2 ?

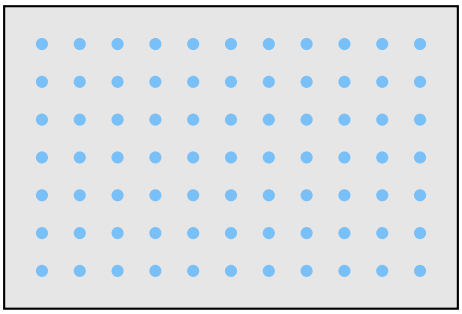
Dans la vie courante, nous comptons en base **10** parce que nous avons dix doigts. Mais un circuit électronique ne possède pas de doigts : il distingue uniquement deux niveaux de tension. C'est pourquoi les machines utilisent le système **binaire** (base 2).

Le système **hexadécimal** (base 16) est une notation compacte du binaire, très utilisée par les programmeurs pour représenter des adresses mémoire, des codes couleur ou des identifiants réseau.



Figure 1.2 – Les deux niveaux logiques d'un circuit numérique.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge



Vue microscopique — millions de transistors

Figure 1.3 – Transistors sur un circuit intégré vus au microscope électronique. Chaque transistor fonctionne comme un interrupteur à deux états (passant/-bloqué), réalisant physiquement le 0 et le 1 du système binaire. Un processeur moderne contient plusieurs milliards de ces composants élémentaires.
Source : YouTube — https://www.youtube.com/watch?v=U885cIhOXBM

Exemple du quotidien : la télécommande infrarouge

Quand vous appuyez sur `Volume+` d'une télécommande, le téléviseur ne reçoit pas le texte « volume ». Il reçoit une **séquence codée de bits** sous forme d'impulsions infrarouges :

`Power` → `10100101 11011010` (protocole NEC, 32 bits)

Chaque touche correspond à un code binaire unique, décodé par le micro-contrôleur du téléviseur.

1.1.2 Principe d'un système positionnel

Système de numération positionnel

Dans un système de numération **positionnel** de base b, la valeur d'un nombre s'obtient en multipliant chaque chiffre par une puissance de b correspondant à sa position.

Pour un nombre à n chiffres $d_{n-1}\, d_{n-2}\, \ldots\, d_1\, d_0$ en base b :

$$N = \sum_{i=0}^{n-1} d_i \times b^i = d_{n-1} \times b^{n-1} + \cdots + d_1 \times b^1 + d_0 \times b^0$$

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

où $d_i \in \{0, 1, \ldots, b-1\}$ est le chiffre de rang i.

</> Application en base 10

Le nombre décimal 572 se décompose comme :

$$572_{10} = 5 \times 10^2 + 7 \times 10^1 + 2 \times 10^0 = 500 + 70 + 2$$

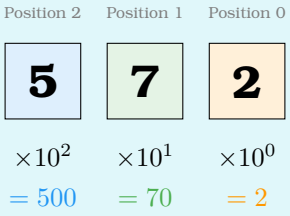


Table 1.1 – Les trois systèmes de numération en informatique.

Système	Base	Symboles	Usage principal
Décimal	10	0–9	Vie courante, affichage utilisateur
Binaire	2	0, 1	Langage natif des circuits
Hexadécimal	16	0–9, A–F	Notation compacte du binaire

1.1.3 Le système binaire (base 2)

Système binaire

Le système **binaire** utilise uniquement deux symboles : **0** et **1**. Chaque position représente une puissance de 2.

Un chiffre binaire est appelé **bit** (*binary digit*). Un groupe de 8 bits forme un **octet** (*byte*).

Table 1.2 – Puissances de 2 à connaître par cœur.

2^0	2^1	2^2	2^3	2^4	2^5	2^6	2^7	2^8	2^9	2^{10}
1	2	4	8	16	32	64	128	256	512	1 024

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Pourquoi ces valeurs reviennent-elles partout ?

- Un octet (8 bits) peut coder $2^8 = 256$ valeurs : composantes couleur RGB (0–255).
- Un mot de 16 bits code $2^{16} = 65\,536$ valeurs : ports réseau TCP/UDP (0–65 535).
- Un mot de 32 bits code environ 4 milliards de valeurs : adresses IPv4.
- $2^{10} = 1\,024 \approx 1\,000$: origine du préfixe « kilo » en informatique (1 Kio).

1.1.3.1 Conversion binaire vers décimal

Conversion de 1101_2 en décimal

$$1101_2 = \underbrace{1}_{\text{bit 3}} \times 2^3 + \underbrace{1}_{\text{bit 2}} \times 2^2 + \underbrace{0}_{\text{bit 1}} \times 2^1 + \underbrace{1}_{\text{bit 0}} \times 2^0 = 8 + 4 + 0 + 1 = \boxed{13_{10}}$$

1	**1**	**0**	**1**	$= 13$
$\times 8$	$\times 4$	$\times 2$	$\times 1$	
$= 8$	$= 4$	$= 0$	$= 1$	

1.1.3.2 Conversion décimal vers binaire

Conversion par divisions successives

Pour convertir un entier décimal N en binaire :

1. Diviser N par 2, noter le **reste** (0 ou 1).
2. Recommencer avec le **quotient** jusqu'à obtenir 0.
3. Lire les restes **de bas en haut**.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

</> Conversion de 25_{10} en binaire

$$
\begin{array}{lll}
25 \div 2 = 12 & \text{reste } \mathbf{1} & \uparrow \text{Lec} \\
12 \div 2 = 6 & \text{reste } \mathbf{0} & \\
6 \div 2 = 3 & \text{reste } \mathbf{0} & \\
3 \div 2 = 1 & \text{reste } \mathbf{1} & \\
1 \div 2 = 0 & \text{reste } \mathbf{1} &
\end{array}
\qquad 25_{10} = 11001_2
$$

Vérification : $1 \times 16 + 1 \times 8 + 0 \times 4 + 0 \times 2 + 1 \times 1 = 25$ ✓

Cas concret : adresse IP en binaire

L'adresse IPv4 `192.168.1.10` est composée de quatre octets. Chaque nombre décimal se convertit en un octet binaire :

192	168	1	10
11000000	10101000	00000001	00001010

Le routeur compare ces bits avec le **masque de sous-réseau** (opération AND bit à bit, cf. section 1.3) pour déterminer si deux machines sont sur le même réseau.

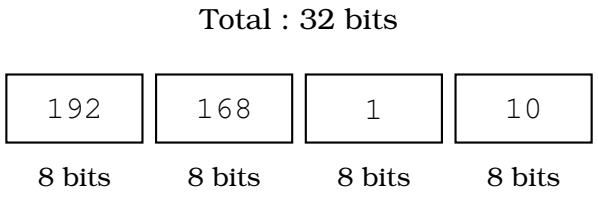


Figure 1.4 – Structure d'une adresse IPv4 : quatre octets (32 bits) écrits en notation décimale pointée. Chaque octet représente une valeur de 0 à 255, soit exactement la plage d'un entier non signé sur 8 bits. Le masque de sous-réseau utilise une opération AND bit à bit pour séparer l'adresse réseau de l'adresse hôte.

Source : Wikimedia Commons, CC BY-SA 4.0 — https://commons.wikimedia.org/wiki/File:IPv4_address_structure_and_writing_systems-en.svg

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

1.1.4 Le système hexadécimal (base 16)

Système hexadécimal

Le système **hexadécimal** utilise seize symboles :

$$0,\ 1,\ 2,\ 3,\ 4,\ 5,\ 6,\ 7,\ 8,\ 9,\ A,\ B,\ C,\ D,\ E,\ F$$

où $A = 10$, $B = 11$, $C = 12$, $D = 13$, $E = 14$, $F = 15$.

Intérêt majeur : un chiffre hexadécimal représente exactement **4 bits** (un *nibble*).

Table 1.3 – Correspondance complète binaire–décimal–hexadécimal.

Bin.	Déc.	Hex.	Bin.	Déc.	Hex.
0000	0	0	1000	8	8
0001	1	1	1001	9	9
0010	2	2	1010	10	A
0011	3	3	1011	11	B
0100	4	4	1100	12	C
0101	5	5	1101	13	D
0110	6	6	1110	14	E
0111	7	7	1111	15	F

</> Pourquoi l'hexadécimal est indispensable

Comparez ces deux écritures du même nombre :

Binaire : `1111 0010 1011 0001` (16 caractères)

Hexadécimal : `F2B1` (4 caractères)

L'hexadécimal est **4 fois plus compact** tout en restant directement convertible en binaire.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

1.1.4.1 Conversion binaire vers hexadécimal et inversement

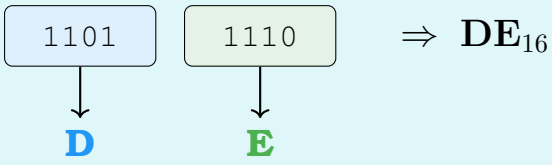
Conversion binaire–hexadécimal

Binaire vers hexadécimal :

1. Regrouper les bits par paquets de 4 **à partir de la droite**.
2. Compléter à gauche par des zéros si nécessaire.
3. Convertir chaque groupe de 4 bits en son chiffre hexadécimal.

Hexadécimal vers binaire : remplacer chaque chiffre hexadécimal par ses 4 bits.

Conversion de 11011110_2 en hexadécimal



Vérification : $D_{16} \times 16 + E_{16} = 13 \times 16 + 14 = 222_{10}$. Et $11011110_2 = 128 + 64 + 16 + 8 + 4 + 2 = 222_{10}$ ✓

Notations en programmation

Base	Notation	Exemple
Binaire	Préfixe `0b`	`0b11001`
Hexadécimal	Préfixe `0x`	`0xFF`, `0x1A3F`
Décimal	Aucun préfixe	`255`, `6719`

En assembleur AVR, les trois formes sont équivalentes :

`LDI R16, 0xFF` = `LDI R16, 0b11111111` = `LDI R16, 255`

1.1.5 Application : les codes couleur RGB

L'une des applications les plus visibles de l'hexadécimal est le codage des couleurs à l'écran.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Codage RGB sur 24 bits

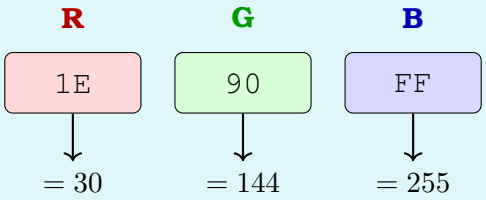
Chaque pixel d'un écran est défini par trois composantes : **Rouge** (R), **Vert** (G) et **Bleu** (B), chacune codée sur un octet (0–255), soit 24 bits par pixel. La notation hexadécimale `#RRGGBB` est universelle en développement web.

Table 1.4 – Exemples de couleurs RGB en hexadécimal.

Couleur	Code Hex	R	G	B
Rouge pur	`#FF0000`	255	0	0
Vert pur	`#00FF00`	0	255	0
Bleu pur	`#0000FF`	0	0	255
Blanc	`#FFFFFF`	255	255	255
Noir	`#000000`	0	0	0
Orange	`#FF9800`	255	152	0

</> Décomposition du code `#1E90FF`

Le code `#1E90FF` (bleu « Dodger Blue ») :



Conversion de $1E_{16}$: $1 \times 16 + 14 = 30_{10}$. Conversion de 90_{16} : $9 \times 16 + 0 = 144_{10}$.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

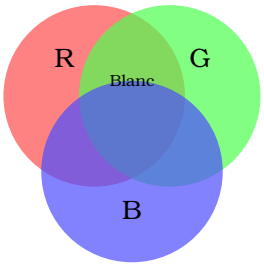


Figure 1.5 – Synthèse additive des couleurs primaires Rouge, Vert et Bleu (RGB). Le mélange des trois à intensité maximale donne le blanc (`#FFFFFF`). Chaque composante est codée sur un octet (0–255) en hexadécimal.
Source : Wikimedia Commons, domaine public — https://commons.wikimedia.org/wiki/File:AdditiveColor.svg

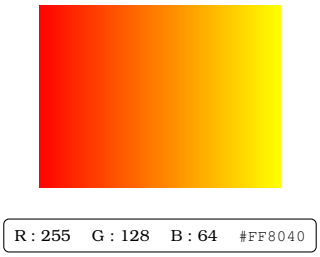
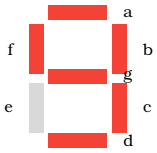


Figure 1.6 – Sélecteur de couleur (*color picker*) tel qu'on le trouve dans les navigateurs web et les logiciels de design (Figma, Photoshop, VS Code). Les valeurs R, G, B sont affichées simultanément en décimal et en hexadécimal, illustrant le lien direct entre ces deux systèmes de numération.

1.1.6 Application : l'afficheur 7 segments

L'afficheur 7 segments est un composant omniprésent : réveils, fours, compteurs, thermomètres numériques. Chaque chiffre affiché correspond à un **motif binaire de 7 bits** indiquant quels segments allumer.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge



Chiffre **9** — Segments actifs : a, b, c, d, f, g

Figure 1.7 – Afficheur 7 segments affichant le chiffre 9. Le segment e (grisé) est éteint. Le microcontrôleur envoie un octet dont chaque bit contrôle un segment.

Table 1.5 – Codes 7 segments pour les chiffres 0–9 (convention `gfedcba`, actif haut).

Chiffre	Segments allumés	Binaire	Hex
0	a, b, c, d, e, f	`0111111`	`0x3F`
1	b, c	`0000110`	`0x06`
2	a, b, d, e, g	`1011011`	`0x5B`
3	a, b, c, d, g	`1001111`	`0x4F`
4	b, c, f, g	`1100110`	`0x66`
5	a, c, d, f, g	`1101101`	`0x6D`
6	a, c, d, e, f, g	`1111101`	`0x7D`
7	a, b, c	`0000111`	`0x07`
8	a, b, c, d, e, f, g	`1111111`	`0x7F`
9	a, b, c, d, f, g	`1101111`	`0x6F`

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

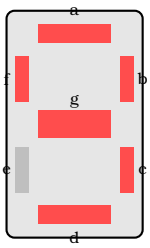


Figure 1.8 – Afficheur 7 segments avec identification des segments a–g. Ce composant est utilisé dans les réveils, fours, compteurs industriels et thermomètres numériques. Chaque segment est piloté par un bit distinct.
Source : Wikimedia Commons, CC BY-SA 3.0 — https://commons.wikimedia.org/wiki/File:7_segment_display_labeled.svg

Lien avec la programmation embarquée

Dans un microcontrôleur AVR, on stocke les motifs dans un tableau :

```
uint8_t digits[] = {0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F
```

Pour afficher le chiffre 5, il suffit d'écrire `PORTB = digits[5];` — le port envoie `0x6D` = 01101101_2, allumant les segments a, c, d, f et g.

1.2 Représentation des entiers

1.2.1 Un même octet, plusieurs significations

Un ordinateur ne stocke que des **bits**. La signification d'un motif binaire dépend entièrement du **contexte d'interprétation** choisi par le programmeur.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Table 1.6 – Un même octet, quatre interprétations différentes.

Interprétation	**Valeur**	**Contexte**
Motif binaire : `11001010`		
Entier non signé	202	Compteur, index
Entier signé (C2)	-54	Température, accélération
Composante couleur	R = 202	Pixel RGB
Code instruction	`CBI`	Opcode AVR

⚠ Point crucial pour le programmeur

La machine stocke des **bits**. C'est le **programmeur** qui décide de leur interprétation via le choix du type de données. En assembleur, il n'y a **pas de types** : c'est vous qui devez savoir si un registre contient un entier signé, non signé, un caractère ou une adresse.

1.2.2 Entiers non signés

Entier non signé sur n bits

Un entier **non signé** (*unsigned*) sur n bits représente uniquement des valeurs positives ou nulles :

$$\boxed{0 \leq x \leq 2^n - 1}$$

Table 1.7 – Plages des entiers non signés et usages concrets.

Bits	**Min**	**Max**	**Usage concret**
4	0	15	Chiffre hexadécimal, nibble
8	0	255	Couleur RGB, caractère ASCII, registre AVR
16	0	65 535	Port TCP/UDP, timer 16 bits
32	0	4 294 967 295	Adresse IPv4, compteur 32 bits

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

1.2.3 Entiers signés et complément à deux

Complément à deux sur n bits

Le **complément à deux** est la méthode standard de représentation des entiers signés dans les processeurs modernes :

- Le **bit de poids fort** (MSB) indique le signe : 0 = positif, 1 = négatif.
- Plage : $\boxed{-2^{n-1} \leq x \leq 2^{n-1} - 1}$

Pour obtenir la représentation de $-x$ (avec $x > 0$) :

1. Écrire x en binaire sur n bits.
2. **Inverser** tous les bits (opération NOT).
3. **Ajouter 1** au résultat.

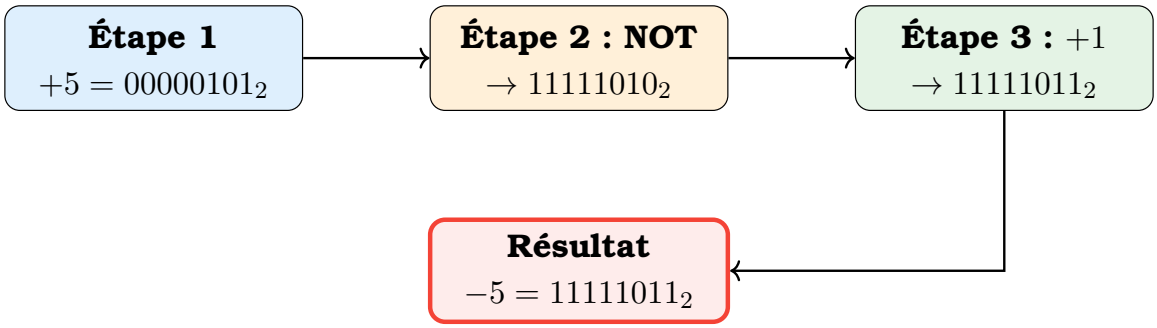


Figure 1.9 – Calcul du complément à deux de -5 sur 8 bits.

Table 1.8 – Comparaison des plages signée et non signée sur 8 bits.

Représentation	Min	Max	Type C
Non signé (8 bits)	0	255	`uint8_t`
Signé C2 (8 bits)	-128	$+127$	`int8_t`

Mêmes bits, valeurs différentes !

Le motif 11111111_2 vaut :

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

- 255 en non signé (`uint8_t`)
- -1 en signé (`int8_t`, complément à deux)

C'est une source classique de bugs en C et en assembleur !

Pourquoi le complément à deux est-il universel ?

L'**addition matérielle** fonctionne de manière identique pour les nombres signés et non signés :

```
    00000101   (+5)
+ 11111011   (−5)
1 00000000   (= 0)
```

Le 9e bit (retenue) est ignoré sur 8 bits

Le processeur utilise **un seul circuit additionneur** pour les deux cas.

1.2.4 Application : capteur de température DS18B20

Capteur DS18B20 : le complément à deux en action

Le capteur **DS18B20**, très courant en domotique et projets Arduino, renvoie la température sur 12 bits en **complément à deux**. Chaque LSB représente 0,0625 °C.

Registre (16 bits)	Valeur	Température
0000 0001 1001 0001	+401	25,0625 °C
0000 0000 0000 0000	0	0 °C
1111 1111 0101 1110	−162	−10,125 °C

Sans le complément à deux, il serait impossible de représenter les températures négatives avec un seul format binaire.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge



Figure 1.10 – Capteur de température DS18B20 de Maxim Integrated (boîtier TO-92, 3 broches : GND, DQ, VDD). Très utilisé en domotique et projets Arduino, il communique via le protocole 1-Wire et transmet la température en complément à deux sur 12 bits.
Source : Wikimedia Commons, CC BY-SA 3.0 — https://commons.wikimedia.org/wiki/File:DS18B20.jpg

1.3 Logique booléenne

1.3.1 Le raisonnement binaire au cœur des machines

Tous les circuits numériques reposent sur des décisions **binaires** : vrai ou faux, 1 ou 0, tension haute ou basse. La **logique booléenne**, formalisée par George Boole (1815–1864), fournit le cadre mathématique de ces opérations.

0	**1**
FAUX / OFF / Bas	VRAI / ON / Haut

Figure 1.11 – Les deux valeurs de la logique booléenne.

La logique booléenne dans la vie quotidienne

- **Interrupteur :** allumé (1) ou éteint (0).
- **Capteur de porte :** ouverte (1) ou fermée (0).
- **Alarme maison :** déclenchée si capteur mouvement **ET** mode armé.
- **Climatisation :** active si température > seuil **OU** mode manuel forcé.
- **Recherche Google :** `"python" AND "tutorial"` filtre les résultats.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

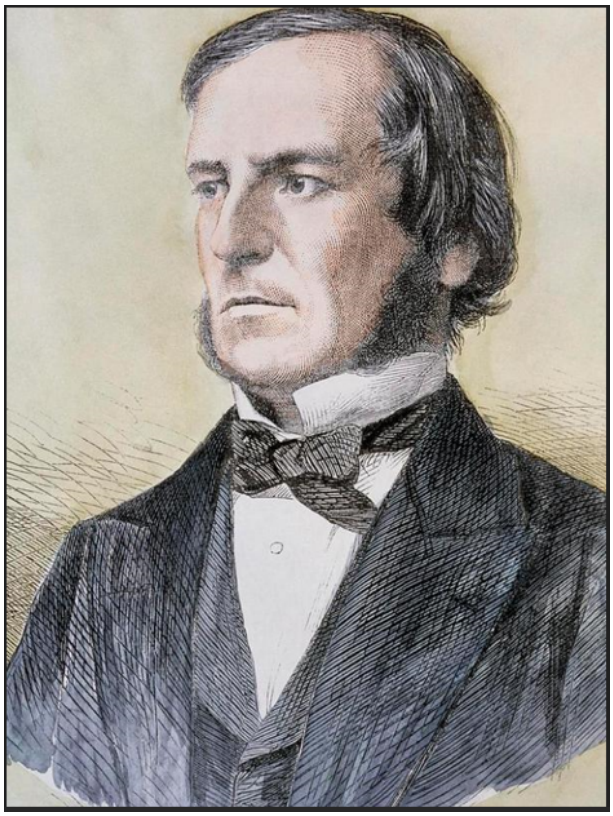


Figure 1.12 – George Boole (1815–1864), mathématicien britannique, fondateur de la logique booléenne. Son algèbre à deux valeurs (0 et 1) constitue le fondement théorique de tous les circuits numériques modernes.
Source : Wikimedia Commons, domaine public — https://commons.wikimedia.org/wiki/File:George_Boole_color.jpg

1.3.2 Les quatre opérations fondamentales

AND	**OR**	**NOT**	**XOR**
Conjonction	**Disjonction**	**Négation**	**Ou exclusif**
$A \wedge B$	$A \vee B$	$\neg A$	$A \oplus B$
« A et B »	« A ou B »	« non A »	« l'un ou l'autre »

Figure 1.13 – Les quatre opérations logiques fondamentales.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

AND ($\wedge$)

A	B	$A \wedge B$
0	0	0
0	1	0
1	0	0
1	1	1

Vrai seulement si A **et** B sont vrais

OR ($\vee$)

A	B	$A \vee B$
0	0	0
0	1	1
1	0	1
1	1	1

Vrai si **au moins** un est vrai

NOT ($\neg$)

A	$\neg A$
0	1
1	0

Inverse la valeur

XOR ($\oplus$)

A	B	$A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

Vrai si A et B sont **différents**

Figure 1.14 – Tables de vérité des quatre opérations logiques fondamentales.

Mnémotechniques pour retenir les opérations

- **AND :** « Toutes les conditions doivent être vraies » — comme un coffre à deux clés.
- **OR :** « Au moins une suffit » — comme deux sonnettes pour une même porte.
- **NOT :** « L'inverse » — comme un interrupteur inverseur.
- **XOR :** « L'un ou l'autre, pas les deux » — comme un va-et-vient d'escalier.

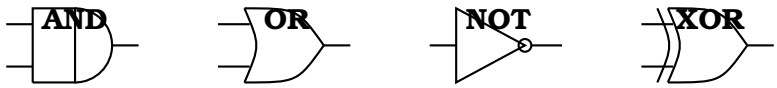


Figure 1.15 – Symboles normalisés (ANSI/IEEE) des portes logiques fondamentales : AND, OR, NOT, XOR. Ces composants élémentaires sont assemblés par milliards dans les processeurs modernes pour réaliser les opérations arithmétiques et logiques.
Source : Wikimedia Commons, domaine public — https://commons.wikimedia.org/wiki/File:Logic-gate-index.png

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

1.3.3 Applications concrètes de la logique

</> Système d'alarme domestique

$$\text{ALARME} = \text{MODE_ARMÉ} \wedge (\text{CAPTEUR_PORTE} \vee \text{CAPTEUR_MVT})$$

L'alarme sonne si le mode est armé **ET** (porte ouverte **OU** mouvement détecté). En assembleur AVR, cela se traduit par des instructions `AND` et `OR` sur des registres.

</> Interrupteur va-et-vient (escalier)

$$\text{LAMPE} = \text{INTER_1} \oplus \text{INTER_2}$$

La lampe s'allume si les deux interrupteurs sont dans des positions **différentes**. C'est la logique XOR : changez n'importe quel interrupteur et l'état bascule.



Figure 1.16 – Schéma électrique d'un interrupteur va-et-vient (deux points de commande). Le comportement correspond exactement à une porte XOR : la lampe s'allume lorsque les deux interrupteurs sont dans des positions différentes.

Source : Wikimedia Commons, domaine public — https://commons.wikimedia.org/wiki/File:Multiway_switching_-_two_locations_(schematic).svg

</> Masque de sous-réseau

Pour déterminer si deux machines sont sur le même réseau, on applique un AND bit à bit entre l'adresse IP et le masque :

```
      192.168.1.10  = 11000000.10101000.00000001.00001010
AND   255.255.255.0 = 11111111.11111111.11111111.00000000
-------------------------------------------------------
    = 192.168.1.0   = 11000000.10101000.00000001.00000000
```

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

1.3.4 Opérations bit à bit en programmation

En programmation et en assembleur, les opérations logiques s'appliquent **bit par bit** sur des mots binaires entiers.

Table 1.9 – Opérateurs bit à bit : symboles et exemples.

Op.	C	AVR	Math	Exemple
AND	`&`	`AND`	$\wedge$	`0b1100 & 0b1010 = 0b1000`
OR	`|`	`OR`	$\vee$	`0b1100 | 0b1010 = 0b1110`
XOR	`^`	`EOR`	$\oplus$	`0b1100 ^ 0b1010 = 0b0110`
NOT	`~`	`COM`	$\neg$	`~0x0F = 0xF0` (8 bits)

</> Masquage : extraire les 4 bits de poids faible

```
  10110110   (valeur = 0xB6 = 182)
AND 00001111   (masque = 0x0F)
-------------
  = 00000110   (résultat = 6)
```

En AVR : `ANDI R16, 0x0F` applique ce masque au registre R16.

</> Forçage de bit : allumer une LED sur le bit 3

Pour mettre le bit 3 à 1 sans modifier les autres bits :

```
  10100001   (PORTB actuel)
OR 00001000   (masque bit 3 = 0x08)
-------------
  = 10101001   (bit 3 allumé, reste inchangé)
```

En AVR : `ORI R16, 0x08` ou `SBI PORTB, 3`.

1.4 Machine à états et cycle d'exécution

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

1.4.1 Concept de machine à états finis

Machine à états finis (FSM)

Une **machine à états finis** (*Finite State Machine*, FSM) est un modèle décrivant un système qui :

- se trouve à tout instant dans l'un d'un nombre **fini d'états** ;
- change d'état selon des **transitions** déterminées par des entrées.

Formellement : FSM $= (S, \Sigma, \delta, s_0, F)$ où S est l'ensemble fini d'états, Σ l'alphabet d'entrée, δ la fonction de transition, s_0 l'état initial, et F les états finaux.

Machines à états dans le quotidien

- **Feu tricolore :** Rouge → Vert → Orange → Rouge.
- **Machine à laver :** Remplissage → Lavage → Rinçage → Essorage → Fin.
- **Distributeur :** Attente → Sélection → Paiement → Distribution.
- **Connexion WiFi :** Déconnecté → Recherche → Authentification → Connecté.
- **Protocole USB :** Idle → Setup → Data → Status → Idle.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

1.4.2 Exemple détaillé : le feu tricolore

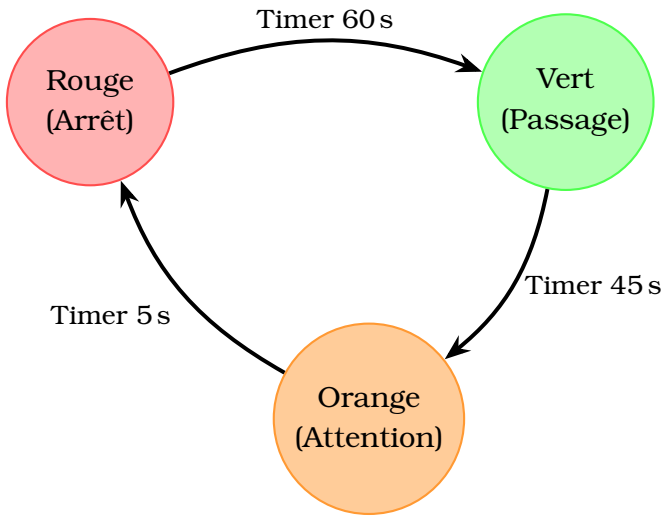


Figure 1.17 – Diagramme d'états d'un feu tricolore. Le système cycle entre trois états, chaque transition étant déclenchée par l'expiration d'un timer.

Table 1.10 – Table de transition du feu tricolore avec codage binaire.

État courant	**Condition**	**État suivant**	**Sortie (R,O,V)**	**Binaire**
Rouge	Timer $\geq$ 60 s	Vert	(1,0,0)	`0b100`
Vert	Timer $\geq$ 45 s	Orange	(0,0,1)	`0b001`
Orange	Timer $\geq$ 5 s	Rouge	(0,1,0)	`0b010`

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

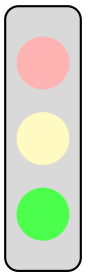


Figure 1.18 – Feu de signalisation tricolore en phase verte. Le séquencement Rouge → Vert → Orange est un exemple classique de machine à états finis, piloté par un microcontrôleur avec timer interne.
Source : Wikimedia Commons, CC BY-SA — https://commons.wikimedia.org/wiki/File:GreenTrafficLight.jpg

1.4.3 Le cycle d'exécution du processeur

Un processeur est lui-même une machine à états sophistiquée. Il traite chaque instruction selon un cycle régulier et déterministe.

Cycle Fetch-Decode-Execute

Le **cycle d'exécution** est la séquence que le processeur répète indéfiniment :

1. **Fetch :** lire l'instruction à l'adresse pointée par le PC.
2. **Decode :** interpréter l'opcode pour identifier l'opération.
3. **Execute :** réaliser l'opération (calcul ALU, transfert, branchement).
4. **Write-back :** enregistrer le résultat dans un registre ou en mémoire.
5. **Update PC :** avancer le compteur de programme.

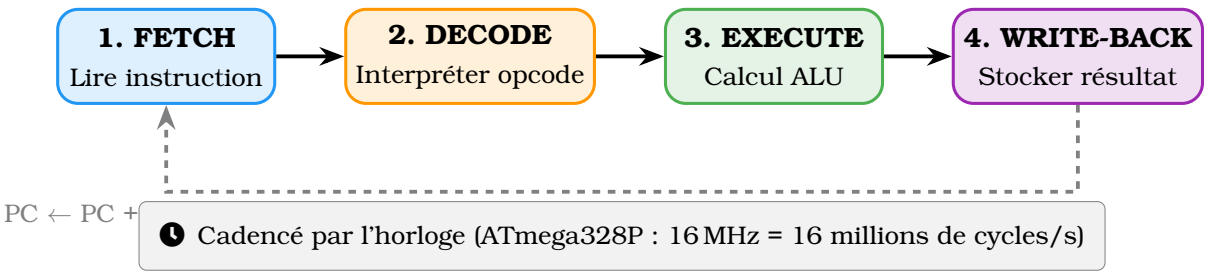


Figure 1.19 – Le cycle Fetch-Decode-Execute-Writeback. Sur un ATmega328P à 16 MHz, ce cycle se répète 16 millions de fois par seconde.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

</> Cycle concret : allumer une LED avec un ATmega328P

Considérons l'instruction `SBI PORTB, 5` (allumer la LED broche 13 d'Arduino) :

Phase	Action	Détail
Fetch	Lit `0x9A2D` en Flash	Opcode de `SBI PORTB, 5`
Decode	Reconnaît l'instruction SBI	Port = PORTB, bit = 5
Execute	Met le bit 5 de PORTB à 1	Tension PB5 passe à 5 V
Write-back	Résultat immédiat	LED allumée
Update PC	PC ← PC + 1	Instruction suivante

Ce processus prend **2 cycles d'horloge** = 125 ns à 16 MHz.

Registres clés du processeur AVR

- **PC** (*Program Counter*) : adresse de la prochaine instruction.
- **IR** (*Instruction Register*) : instruction en cours de décodage.
- **SREG** (*Status Register*) : drapeaux Z, C, N, V, S, H.
- **SP** (*Stack Pointer*) : sommet de la pile en SRAM.
- **R0–R31** : 32 registres généraux de 8 bits.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

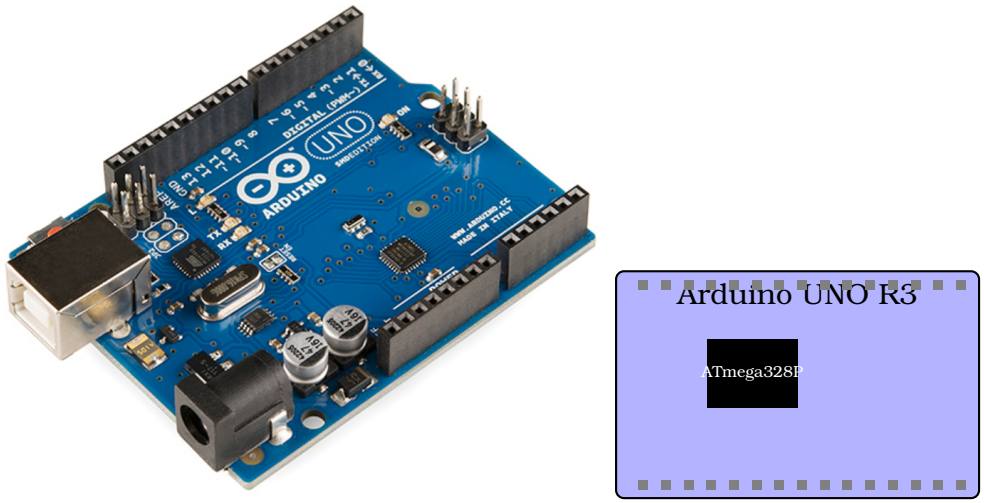


Figure 1.20 – Carte Arduino UNO R3 équipée du microcontrôleur ATmega328P (puce noire au centre). Ce processeur AVR 8 bits cadencé à 16 MHz exécute le cycle Fetch-Decode-Execute 16 millions de fois par seconde. C'est la plateforme de référence pour les travaux pratiques de ce cours.

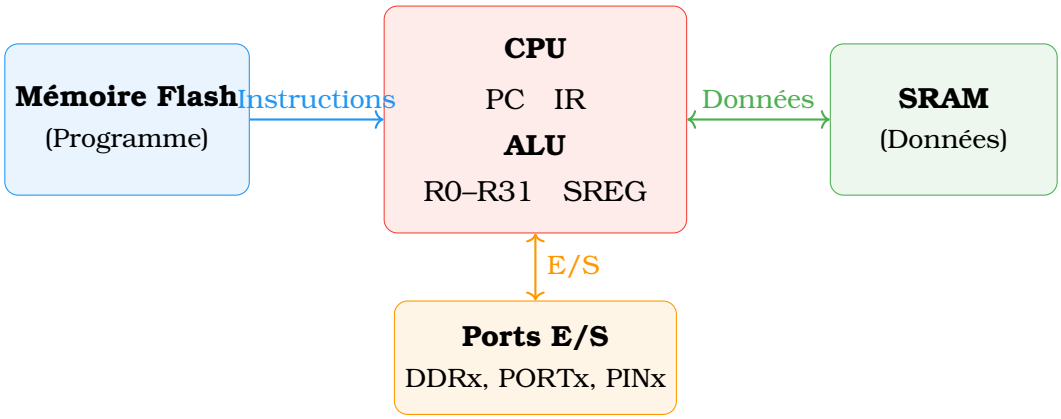


Figure 1.21 – Architecture simplifiée d'un microcontrôleur AVR : la mémoire Flash fournit les instructions, le CPU les exécute via l'ALU et les registres, la SRAM stocke les données, et les ports E/S communiquent avec l'extérieur.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

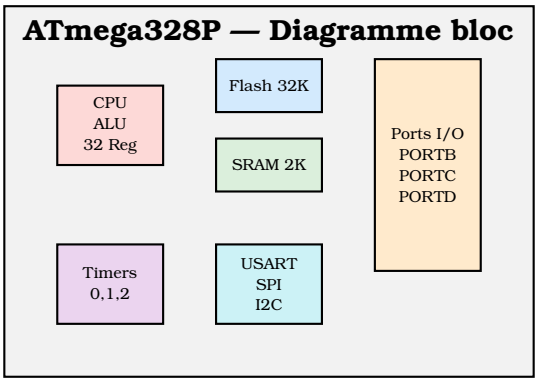


Figure 1.22 – Diagramme bloc de l'ATmega328P montrant l'architecture Harvard modifiée : mémoire Flash (programme) et SRAM (données) séparées, ALU 8 bits, 32 registres généraux, compteur de programme, registre d'état (SREG), ports d'E/S, timers et interfaces de communication (USART, SPI, I2C). Ce schéma sera détaillé au chapitre 1.
Source : Microchip Technology Datasheet — https://ww1.microchip.com/downloads/en/DeviceDoc/ATmega48A-PA-88A-PA-168A-PA-328-P-DS-DS40002061B.pdf

Synthèse du chapitre 0

Ce chapitre a posé les quatre piliers nécessaires à l'étude de l'architecture et de l'assembleur :

1. **Systèmes de numération** : le binaire est le langage natif des machines ; l'hexadécimal en est la notation compacte.
2. **Représentation des entiers** : un même motif binaire peut être interprété différemment. Le complément à deux code les négatifs avec un seul additionneur.
3. **Logique booléenne** : AND, OR, NOT et XOR sont les briques de tout circuit. Le masquage et le forçage de bits sont essentiels en bas niveau.
4. **Machine à états et cycle d'exécution** : un processeur répète indéfiniment le cycle Fetch-Decode-Execute, cadencé par une horloge.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

*Ces fondements seront mobilisés dans **chaque chapitre suivant** : les registres AVR stockent des octets (ch. 1), les instructions manipulent des bits (ch. 2–3), la pile utilise l'adressage binaire (ch. 4), et les E/S reposent sur le masquage logique (ch. 5).*

Formules et repères essentiels

Concept	**Formule ou repère**
Valeur positionnelle	$N = \sum_{i=0}^{n-1} d_i \times b^i$
Plage non signée (n bits)	$[0\,;\ 2^n - 1]$
Plage signée C2 (n bits)	$[-2^{n-1}\,;\ 2^{n-1} - 1]$
Complément à deux de x	$\text{NOT}(x) + 1$
AND (masquage)	Isole les bits à 1 dans le masque
OR (forçage)	Met à 1 les bits à 1 dans le masque
XOR (basculement)	Inverse les bits à 1 dans le masque
1 chiffre hex	$= 4$ bits (1 nibble)
1 octet	$= 8$ bits $= 2$ chiffres hex

1.5 Travaux dirigés du chapitre 0

Travaux dirigés : Du bit au système

5 exercices progressifs ancrés dans des situations réelles — réseau, développement web, systèmes embarqués, cybersécurité

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

TD 1 — Le monde des adresses : IP, MAC et couleurs RGB

Contexte : Vous êtes technicien réseau junior. Votre responsable vous demande d'analyser les configurations d'équipements et de comprendre le codage des données réseau et graphiques.

Situation 1 — Configuration réseau d'un serveur

Le serveur de l'entreprise a l'adresse IP `192.168.10.254` avec le masque `255.255.255.0`.

a) Convertissez chaque octet de l'adresse IP en binaire sur 8 bits.

b) Convertissez le masque de sous-réseau en binaire. Combien de bits sont à 1 ? Quelle est la notation CIDR correspondante (`/xx`) ?

c) Un collègue vous donne l'adresse `192.168.10.100`. En appliquant un AND bit à bit entre cette adresse et le masque, déterminez si elle appartient au même réseau que le serveur.

d) L'adresse de broadcast du réseau est obtenue en mettant tous les bits « hôte » à 1. Quelle est-elle ?

Situation 2 — Analyse d'une adresse MAC

La carte réseau d'un poste affiche l'adresse MAC `00:1A:2B:3C:4D:5E`.

a) Combien d'octets compose cette adresse ? Combien de bits au total ?

b) Convertissez le premier octet (`00`) et le dernier (`5E`) en binaire.

c) Le bit de poids faible du premier octet indique si l'adresse est unicast (0) ou multicast (1). Cette adresse est-elle unicast ou multicast ?

d) Le deuxième bit de poids faible indique si l'adresse est universelle (0) ou locale (1). Qu'en est-il pour cette adresse ?

Situation 3 — Design d'interface web

Vous développez une interface web et le designer vous fournit la palette suivante :

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

- Couleur principale : `#1E90FF` (Dodger Blue)
- Couleur d'accentuation : `#FF6B35` (Orange vif)
- Fond : `#F5F5F5` (Gris très clair)

a) Pour chaque couleur, décomposez le code hexadécimal en trois composantes R, G, B (en décimal).

b) La couleur `#1E90FF` a-t-elle plus de rouge, de vert ou de bleu ?

c) Calculez la luminosité approximative de chaque couleur avec la formule simplifiée : $L = 0.3 \times R + 0.6 \times G + 0.1 \times B$. Quelle couleur est la plus lumineuse ?

d) Vous devez créer une version « assombrie » de `#1E90FF` en divisant chaque composante par 2. Quel est le nouveau code hexadécimal ?

TD 2 — Capteurs IoT et données signées

Contexte : Vous développez une application IoT qui collecte des données de capteurs. Comprendre la représentation des données signées et non signées est crucial pour interpréter correctement les mesures.

Situation 1 — Capteur de température DS18B20

Le capteur DS18B20 renvoie la température sur 12 bits en complément à deux. Chaque LSB = 0,0625 °C. Le registre 16 bits lu contient la valeur avec les 4 bits de poids fort à 0 (ou 1 si négatif).

a) Le capteur renvoie `0x0191`. Convertissez en binaire, puis en décimal. Quelle est la température mesurée ?

b) Le capteur renvoie `0xFF5E`. Le MSB étant à 1, c'est une valeur négative. Appliquez la procédure inverse du C2 (NOT + 1) sur les 12 bits significatifs pour trouver la valeur absolue. Quelle est la température ?

c) Quelle est la plage de températures mesurables avec ce format 12 bits signé ?

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

d) Votre collègue lit la valeur brute `0x0000` et dit « le capteur est cassé ». A-t-il raison ?

Situation 2 — Accéléromètre 3 axes

Un accéléromètre MEMS (type ADXL345) renvoie l'accélération sur chaque axe en **complément à deux sur 16 bits**. La sensibilité est de 256 LSB/g (mode ±2g).

a) L'axe Z (vertical) renvoie `0x0100` = 256_{10}. Quelle accélération en g cela représente-t-il ? Est-ce cohérent pour un capteur posé à plat ?

b) L'axe X renvoie `0xFFE0`. Convertissez en signé (C2). Quelle accélération en g ?

c) Calculez la représentation C2 sur 16 bits de l'accélération $-0.5g$.

d) Quelle est la plage d'accélérations mesurables avec ce format ?

Situation 3 — Compteur d'énergie

Un compteur électrique intelligent utilise un registre 32 bits non signé pour stocker la consommation en Wh.

a) Quelle est la valeur maximale stockable ? Convertissez en kWh puis en MWh.

b) Une maison consomme en moyenne 10 kWh/jour. Au bout de combien de jours le compteur débordera-t-il ?

c) Si le compteur passait à 64 bits, combien de temps avant débordement avec la même consommation ?

d) Le compteur actuel affiche `0x000F4240`. Convertissez en décimal, puis en kWh.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

TD 3 — Logique booléenne pour la sécurité et le contrôle d'accès

Contexte : Vous concevez la logique de contrôle d'un système de sécurité pour un datacenter. La compréhension des opérations booléennes est essentielle pour définir les règles d'accès et d'alerte.

Situation 1 — Système d'alarme datacenter

Le système de sécurité possède les entrées suivantes :

- B : Badge valide présenté (1 = oui)
- C : Code PIN correct (1 = oui)
- H : Horaire autorisé (1 = oui, entre 7h et 20h)
- A : Mode admin activé (1 = oui, bypass horaire)

La porte s'ouvre si : Badge valide **ET** Code correct **ET** (Horaire autorisé **OU** Mode admin)

a) Écrivez l'expression booléenne : PORTE $= ?$

b) Construisez la table de vérité complète (16 lignes).

c) Un employé a un badge valide, tape le bon code à 22h sans mode admin. La porte s'ouvre-t-elle ?

d) L'administrateur système a le mode admin activé mais oublie son code. Peut-il entrer ?

Situation 2 — Règles de pare-feu simplifiées

Vous configurez un pare-feu avec les règles suivantes pour le trafic entrant :

- Autoriser si : (port 80 **OU** port 443) **ET** (source = réseau interne **OU** IP whitelist)
- Bloquer sinon

Variables : P_{80} (port 80), P_{443} (port 443), I (réseau interne), W (whitelist)

a) Écrivez l'expression : AUTORISER $= ?$

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

b) Un paquet arrive sur le port 443 depuis une IP externe non whitelistée. Est-il autorisé ?

c) Simplifiez l'expression si on sait que le réseau interne est toujours whitelisté ($I \Rightarrow W$).

d) Appliquez la loi de De Morgan pour exprimer la condition de blocage.

Situation 3 — Détection d'intrusion

Un système de détection d'intrusion génère une alerte si :

- Tentatives de connexion échouées ≥ 5 (F) **ET** même IP (S)
- **OU** Scan de ports détecté (P)
- **OU** Payload suspect (M) **ET** connexion depuis pays à risque (R)

a) Écrivez l'expression : ALERTE $= ?$

b) Construisez la table de vérité pour les 32 combinaisons possibles. (Indication : utilisez un tableau synthétique en regroupant les cas où ALERTE = 1)

c) Un attaquant utilise 10 IP différentes pour ses tentatives. L'alerte est-elle déclenchée par la première condition ?

d) Proposez une amélioration de la formule pour détecter aussi les attaques distribuées.

TD 4 — Manipulation de registres et configuration matérielle

Contexte : Vous programmez un microcontrôleur Arduino (ATmega328P) pour un projet de domotique. La manipulation des registres via des opérations bit à bit est fondamentale.

Situation 1 — Contrôle de LEDs sur PORTB

Le port B de l'ATmega328P contrôle 8 broches (PB0 à PB7). Chaque bit

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

du registre `PORTB` contrôle une LED (1 = allumée).
État actuel : `PORTB = 0b00110100` = `0x34`

a) Quelles LEDs sont actuellement allumées ? (numérotez de 0 à 7)

b) Écrivez l'opération pour **allumer** la LED 0 **sans modifier** les autres. Donnez le résultat.

c) Écrivez l'opération pour **éteindre** la LED 4 **sans modifier** les autres. Donnez le résultat.

d) Écrivez l'opération pour **inverser** l'état de la LED 2 (toggle). Donnez le résultat.

e) Vous voulez allumer les LEDs 0, 1, 6, 7 et éteindre toutes les autres. Quelle valeur écrire directement dans `PORTB` ?

Situation 2 — Lecture de capteurs sur PINC

Le port C est configuré en entrée et lit l'état de 8 capteurs. Le registre `PINC` contient `0b10110010` = `0xB2`.
Structure des capteurs :

- Bits 0-3 : 4 capteurs de contact (1 = contact)
- Bits 4-5 : Capteur de luminosité (valeur 0-3)
- Bits 6-7 : Capteur de température (valeur 0-3)

a) Quels capteurs de contact sont actifs ?

b) Extrayez la valeur du capteur de luminosité (bits 4-5). Utilisez un masque ET puis un décalage logique.

c) Extrayez la valeur du capteur de température (bits 6-7).

d) Écrivez une expression qui teste si **au moins un** capteur de contact est actif.

e) Le capteur de luminosité renvoie 0 = nuit, 1 = faible, 2 = moyen, 3 = jour. Quelle est la luminosité actuelle ?

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Situation 3 — Registre de configuration UART

Vous configurez le registre de contrôle UART `UCSRB` de l'ATmega328P. Voici la signification des bits :

Bit	Nom	Fonction
7	RXCIE	Interruption réception activée
6	TXCIE	Interruption transmission activée
5	UDRIE	Interruption buffer vide activée
4	RXEN	Récepteur activé
3	TXEN	Transmetteur activé
2	UCSZ2	Format données (bit 2)
1	RXB8	9e bit reçu (lecture seule)
0	TXB8	9e bit à transmettre

État actuel : `UCSRB = 0x00`

a) Écrivez le code pour activer le récepteur ET le transmetteur (bits 3 et 4) sans modifier les autres bits.

b) Ajoutez l'activation de l'interruption de réception (bit 7). Donnez la valeur finale de `UCSRB`.

c) Sans tout réécrire, comment désactiver uniquement le transmetteur (bit 3) ?

d) Écrivez une expression qui teste si le transmetteur est actif.

TD 5 — Conception de machines à états pour systèmes embarqués

Contexte : Vous concevez des systèmes embarqués et devez modéliser leur comportement par des machines à états finis. Cette approche est fondamentale en développement firmware et automates industriels.

Situation 1 — Connexion WiFi d'un objet connecté

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Modélisez la connexion WiFi d'un capteur IoT avec les états suivants :

- **IDLE** : En veille, WiFi désactivé
- **SCANNING** : Recherche des réseaux disponibles
- **CONNECTING** : Tentative de connexion au réseau configuré
- **AUTHENTICATING** : Échange des clés de sécurité (WPA2)
- **CONNECTED** : Connexion établie, IP obtenue
- **ERROR** : Échec de connexion

Transitions :

- IDLE $\xrightarrow{\text{bouton}}$ SCANNING
- SCANNING $\xrightarrow{\text{réseau trouvé}}$ CONNECTING
- SCANNING $\xrightarrow{\text{timeout 10s}}$ ERROR
- CONNECTING $\xrightarrow{\text{réponse AP}}$ AUTHENTICATING
- AUTHENTICATING $\xrightarrow{\text{succès}}$ CONNECTED
- AUTHENTICATING $\xrightarrow{\text{échec}}$ ERROR
- CONNECTED $\xrightarrow{\text{déconnexion}}$ IDLE
- ERROR $\xrightarrow{\text{retry}}$ SCANNING
- ERROR $\xrightarrow{\text{abandon}}$ IDLE

a) Dessinez le diagramme d'états complet avec cercles et flèches annotées.

b) Combien de bits faut-il pour encoder les 6 états en binaire ?

c) Proposez un codage binaire pour chaque état.

d) Ajoutez un état **RECONNECTING** pour les déconnexions temporaires (différent de ERROR).

Situation 2 — Protocole de communication série

Vous implémentez un protocole de communication simple avec un capteur. Les trames ont le format :

```
[START] [CMD] [DATA] [CHECKSUM] [END]
0xAA  [1 octet][N octets][1 octet] 0x55
```

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Modélisez la réception d'une trame avec une FSM.

a) Identifiez les états nécessaires pour parser cette trame.

b) Dessinez le diagramme d'états avec les transitions (octet reçu, valeur attendue).

c) Que se passe-t-il si on reçoit `0x55` alors qu'on attend des données ?

d) Ajoutez un mécanisme de timeout : retour à l'état initial si aucun octet pendant 100 ms.

Situation 3 — Distributeur de café connecté

Concevez la machine à états d'un distributeur de café intelligent avec :
Entrées :

- Boutons : Café court, Café long, Cappuccino, Annuler
- Capteurs : Niveau eau, Niveau café, Gobelet présent, Paiement validé (NFC)
- Timer : Durées de préparation

Sorties :

- Actionneurs : Pompe eau, Moulin, Vapeur
- Affichage : Messages utilisateur
- LEDs : État machine

Contraintes :

- Ne pas démarrer si niveau eau ou café insuffisant
- Attendre le gobelet avant de servir
- Permettre l'annulation à tout moment (sauf pendant le service)

a) Listez tous les états nécessaires (minimum 8).

b) Dessinez le diagramme d'états complet.

c) Créez la table de transition (état actuel, entrée, état suivant, sortie).

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

d) Proposez un encodage des états sur 4 bits avec une logique (ex. : bit 3 = erreur).

e) Comment gérer les erreurs (réservoir vide pendant la préparation) ?

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

1.6 Quiz du chapitre 0 : testez vos connaissances

Vrai/Faux QCM Pratique

Répondez sans regarder le cours. Ces questions simulent des situations réelles rencontrées en entreprise.

1 QCM

Un développeur web définit une couleur avec le code `#FF8000`. Quelles sont les composantes RGB en décimal ?

A. R=255, G=128, B=0 (orange)

B. R=255, G=80, B=00 (rouge-orange)

C. R=15, G=8, B=0 (brun foncé)

D. R=128, G=0, B=255 (violet)

2 VRAI/FAUX

Une adresse IPv4 comme `192.168.1.1` occupe exactement 32 bits en mémoire (4 octets).

☐ **Vrai** ☐ **Faux**

Indice : chaque nombre (0–255) est un octet.

3 QCM

Un capteur de température renvoie la valeur `0xFF` (8 bits). Si on l'interprète comme signé (C2), quelle température représente-t-il si 1 LSB = 1 °C ?

A. +255 °C

B. +127 °C

C. -1 °C

D. 0 °C

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

4 VRAI/FAUX

Pour éteindre un bit spécifique dans un registre sans affecter les autres, on utilise l'opération OR avec un masque.

☐ **Vrai** ☐ **Faux**

Indice : OR met des bits à 1. Pour mettre à 0, quelle opération ?

5 QCM

Dans un système d'alarme, la sirène sonne si : `mode_armé AND (porte_ouverte OR mouvement)`. Si le mode n'est pas armé, la sirène peut-elle sonner ?

A. Oui, si la porte est ouverte

B. Oui, si un mouvement est détecté

C. Non, jamais

D. Cela dépend du type de capteur

6 VRAI/FAUX

L'opération XOR permet de basculer (toggle) un bit : si le bit est à 0 il passe à 1, et vice versa.

☐ **Vrai** ☐ **Faux**

Indice : calculez $0 \oplus 1$ *et* $1 \oplus 1$.

7 PRATIQUE

Vous devez configurer un registre de LEDs. État actuel : `0b01010101`. Écrivez les opérations pour :

a) Allumer la LED 1 (bit 1) : `PORTB = PORTB` _____

b) Éteindre la LED 6 (bit 6) : `PORTB = PORTB` _____

c) Inverser la LED 0 (bit 0) : `PORTB = PORTB` _____

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

8 QCM

Un processeur ARM cadencé à 1 GHz exécute 1 instruction par cycle. Combien d'instructions exécute-t-il en 1 microseconde ?

A. 1

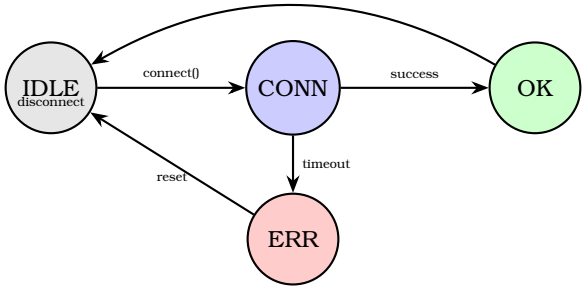
B. 1 000

C. 1 000 000

D. 1 000 000 000

9 PRATIQUE

Identifiez l'état suivant dans cette machine à états de connexion réseau :



État actuel : **CONN**. Événement : **timeout**. État suivant : _____

État actuel : **OK**. Événement : **disconnect**. État suivant : _____

État actuel : **ERR**. Événement : **connect()**. État suivant : _____

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

10 PRATIQUE

Défi réseau : Vous analysez un paquet avec Wireshark. L'adresse IP source est affichée comme `C0.A8.0A.FE` (notation hexadécimale). Convertissez-la en notation décimale pointée standard.

`C0` = _____ (décimal)

`A8` = _____ (décimal)

`0A` = _____ (décimal)

`FE` = _____ (décimal)

Adresse IP : _____

Bonus : Cette adresse appartient-elle à un réseau privé (RFC 1918) ?

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Récapitulatif : Ce que vous devez maîtriser

Numération — Applications

- Adresses IP : 4 octets, notation décimale
- Adresses MAC : 6 octets, notation hex
- Couleurs RGB : 3 octets, `#RRGGBB`
- Masques : opérations AND bit à bit
- 1 chiffre hex = 4 bits = 1 nibble

Représentation — Capteurs

- Non signé : compteurs, index, tailles
- Signé (C2) : températures, accélérations
- Même bits $\neq$ même valeur !
- Overflow : comportement circulaire

Logique — Sécurité & config

- AND : conditions multiples obligatoires
- OR : conditions alternatives
- XOR : toggle, chiffrement simple
- Masquage : extraction de champs
- Forçage : configuration de bits

FSM — Protocoles & systèmes

- États finis + transitions déterministes
- Connexions réseau : IDLE $\rightarrow$ CONN $\rightarrow$ OK
- Protocoles : parsing de trames
- Cycle CPU : Fetch-Decode-Execute

Compétences clés pour le métier IT :

Lire un dump hexadécimal • Configurer des registres matériels • Analyser des trames réseau Interpréter des données de capteurs • Concevoir des automates • Déboguer au niveau bit

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Fin du Chapitre 0 — Fondements préliminaires

Vous maîtrisez maintenant les bases indispensables :

Numération • Représentation • Logique • Machine à états

Chapitre suivant : Architecture des microcontrôleurs AVR

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

CHAPITRE II

Introduction à l'architecture des microcontrôleurs AVR

Chapitre 2
Introduction aux microcontrôleurs AVR

Comprendre la machine avant de la programmer

4 heures | Fondements théoriques | Systèmes embarqués | Architecture matérielle

Mise en situation : une journée ordinaire

Imaginez votre journée typique :

1. **6h30** — Votre **réveil** sonne. *Comment sait-il qu'il est 6h30 ?* Un petit composant électronique compte les secondes depuis minuit et compare avec l'heure programmée. Quand les deux correspondent, il active le buzzer.
2. **6h35** — Votre **cafetière programmable** commence à chauffer l'eau. *Comment sait-elle qu'il faut démarrer maintenant ?* Le même type de composant surveille l'heure, vérifie qu'il y a de l'eau dans le réservoir, chauffe à la bonne température et s'arrête automatiquement.

3. **7h00** — Vous montez dans votre **voiture**. Le moteur ne démarre pas immédiatement : le véhicule vérifie d'abord la pression d'huile, la température du moteur, l'état de la batterie... *Qui fait toutes ces vérifications ?*
4. **7h15** — En roulant, votre voiture **freine brusquement** sur une route mouillée. Les roues ne se bloquent pas grâce à l'**ABS**. *Qui surveille la vitesse de chaque roue 100 fois par seconde et ajuste la pression de freinage ?*
5. **8h00** — Vous badgez à l'entrée du bâtiment. La **porte s'ouvre** automatiquement. *Qui a lu le badge, vérifié l'autorisation et commandé l'ouverture en moins d'une seconde ?*

La réponse est toujours la même :
un microcontrôleur.

Un composant minuscule, souvent invisible, qui prend des milliers de décisions par seconde dans chacun de ces objets.

Pourquoi ce chapitre est fondamental

Ce cours porte sur la **programmation assembleur**. Mais avant d'écrire la moindre ligne de code, une question s'impose : *pour qui écrit-on ce code ?*

Quand vous écrivez un programme en Python ou en Java, vous ne vous souciez pas vraiment de la machine : le langage s'occupe des détails.
En assembleur, c'est l'inverse. **Chaque instruction que vous écrirez correspondra à une action physique précise** du microcontrôleur :

- déplacer un nombre d'un tiroir à un autre (registre) ;
- lire une valeur venant d'un capteur (port d'entrée) ;
- allumer une LED en envoyant du courant (port de sortie) ;
- compter le temps qui passe (timer) ;
- additionner deux nombres (ALU).

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Pour comprendre **pourquoi** une instruction fait ce qu'elle fait, il faut connaître **la machine**. C'est exactement l'objet de ce chapitre.

Structure interne | **Mémoires (3 types)** | **Registres (32 tiroirs)** | **Interfaces avec l'extérieur** | **Comment il exécute le code**

◎ Objectifs du chapitre

À l'issue de ce chapitre, vous serez capable de :

1. **Expliquer** avec vos propres mots ce qu'est un microcontrôleur et à quoi il sert.
2. **Donner** au moins 5 exemples d'objets du quotidien qui en contiennent.
3. **Distinguer** un microcontrôleur d'un microprocesseur en expliquant pourquoi on utilise l'un plutôt que l'autre.
4. **Dessiner** de mémoire un schéma simplifié de l'architecture AVR avec ses composants principaux.
5. **Expliquer** le rôle de chaque composant (CPU, ALU, registres, Flash, SRAM, EEPROM).
6. **Décrire** comment le processeur exécute une instruction, étape par étape.
7. **Installer** un environnement de développement et faire clignoter une LED.

🔑 Prérequis

Ce chapitre suppose acquis les fondements du chapitre 0 :

- ✔ Systèmes de numération (binaire, hexadécimal) — *vous savez convertir* `0xFF` *en 255*
- ✔ Représentation des entiers (complément à deux) — *vous savez que* `11111111` *peut valoir 255 ou* -1
- ✔ Opérations logiques (AND, OR, NOT, XOR) — *vous savez faire un masquage de bits*

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

✔ Notion de machine à états — *vous connaissez le feu tricolore comme FSM*

Si ce n'est pas le cas, relisez le chapitre 0 avant de continuer. Chaque notion ci-dessus sera utilisée concrètement dans ce chapitre.

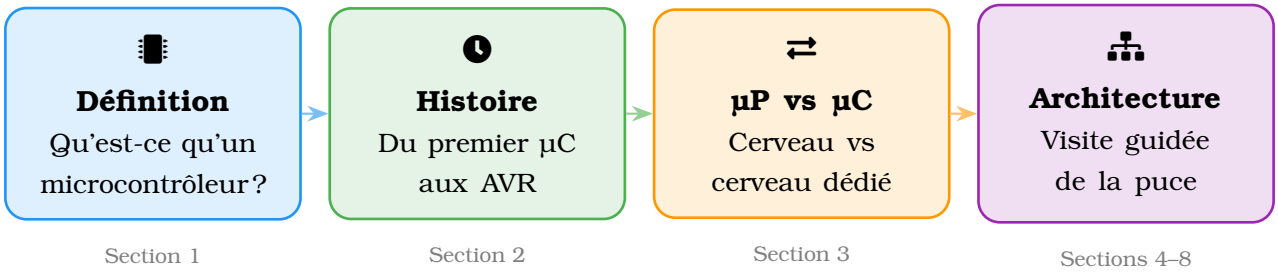


Figure 2.1 – Organisation du chapitre : de la découverte du microcontrôleur à la visite détaillée de son architecture interne.

2.1 Qu'est-ce qu'un microcontrôleur ?

2.1.1 Commençons par une analogie

Le microcontrôleur, c'est comme un employé polyvalent

Imaginez une petite entreprise qui n'a qu'un seul employé. Cet employé doit tout faire :

- **lire** le courrier (= lire les capteurs) ;
- **réfléchir** et prendre des décisions (= calculer avec le processeur) ;
- **se souvenir** des consignes (= stocker le programme en mémoire) ;
- **noter** des résultats temporaires sur un bloc-notes (= utiliser la RAM) ;
- **agir** : allumer la lumière, ouvrir la porte, envoyer un message (= commander les sorties).

Un microcontrôleur, c'est exactement cela : **un employé miniature intégré dans une seule puce**, qui fait tout tout seul.

Par contraste, un microprocesseur, c'est un employé très intelligent mais

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

qui a besoin de collègues (mémoire externe, circuits d'entrée/sortie) pour fonctionner.

2.1.2 Définition formelle

Microcontrôleur (*Microcontroller Unit* — MCU)

Un **microcontrôleur** est un **circuit intégré programmable** qui regroupe sur une **seule puce de silicium** tous les éléments nécessaires pour constituer un petit ordinateur autonome. Il est conçu pour piloter un système embarqué de manière **autonome**, **compacte**, **réactive** et **économe en énergie**.

Microcontrôleur = un ordinateur miniature sur une seule puce

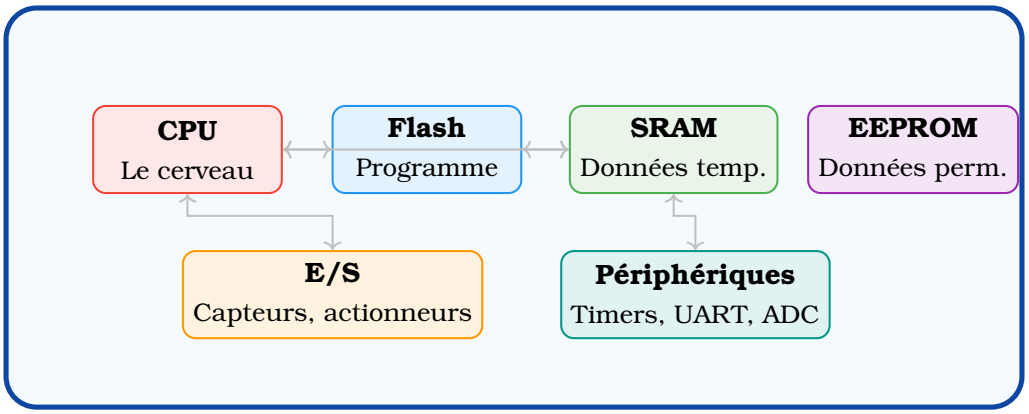


Figure 2.2 – Synoptique d'un microcontrôleur : tous les composants sur une seule puce.

Taille réelle d'un microcontrôleur

Un microcontrôleur comme l'ATmega328P mesure environ **3 cm de long** dans son boîtier DIP-28. Mais la puce de silicium à l'intérieur ne fait que **quelques millimètres carrés** — à peine la taille d'une tête d'épingle !
Et pourtant, cette minuscule surface contient :

- un processeur complet ;
- 32 768 octets de mémoire programme ;

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

- 2 048 octets de RAM ;
- 23 broches d'entrée/sortie ;
- 3 timers, 1 convertisseur analogique-numérique, des interfaces de communication. . .

Le tout consommant moins de 15 mW en fonctionnement normal.

2.1.3 Le microcontrôleur dans les objets du quotidien

Pour bien comprendre ce qu'est un microcontrôleur, examinons en détail comment il fonctionne dans des objets que vous utilisez tous les jours.

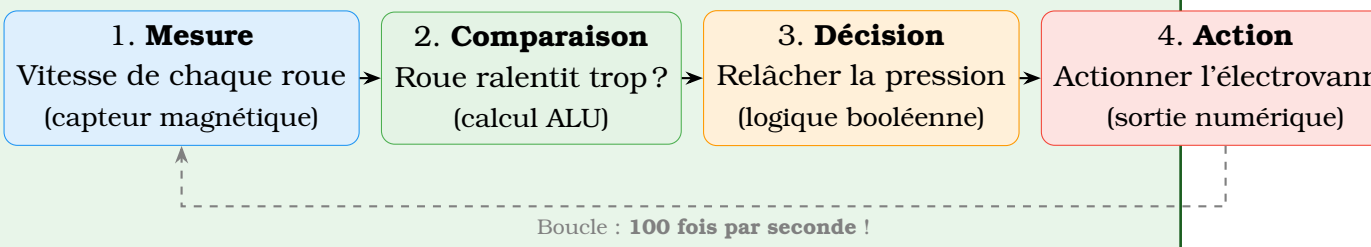
Quand vous réglez votre réveil sur 6h30 :

1. Vous appuyez sur des boutons pour entrer l'heure. Le µC **lit l'état des boutons** (entrées numériques).
2. Le µC **stocke** l'heure de réveil en mémoire (EEPROM — pour ne pas la perdre si on débranche le réveil).
3. Un **quartz** vibre exactement 32 768 fois par seconde. Le µC **compte ces vibrations** grâce à un timer. Après 32 768 vibrations = 1 seconde écoulée.
4. Chaque seconde, le µC met à jour l'heure et **l'affiche** sur l'écran à 7 segments (sorties).
5. Quand l'heure actuelle = l'heure programmée, le µC **active le buzzer** (sortie).

Bilan : Un seul composant, qui coûte moins de 0.50 €, gère toute la logique d'un réveil.

L'ABS (*Anti-lock Braking System*) empêche les roues de se bloquer lors d'un freinage brusque :

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge



Le µC exécute cette boucle **100 fois par seconde** pour chaque roue. C'est impossible pour un humain, trivial pour un microcontrôleur à 16 MHz.

Quand vous présentez votre badge :

1. Le lecteur RFID **lit le numéro** du badge (communication série SPI ou I^2C).
2. Le µC **compare** ce numéro avec la liste des numéros autorisés (stockés en EEPROM ou Flash).
3. Si le numéro est dans la liste **ET** l'heure est dans la plage autorisée → le µC active un relais qui ouvre la porte.
4. Sinon → LED rouge + buzzer d'erreur.
5. Le µC **enregistre** l'événement (heure, badge, résultat) dans un journal en mémoire.

Vous reconnaissez ici la logique booléenne du chapitre 0 : l'accès est autorisé si `badge_valide AND heure_ok`.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Table 2.1 – Microcontrôleurs dans le quotidien : que fait-il exactement ?

Objet	**Entrées**	**Sorties**	**Traitement**
Réveil	Boutons, quartz	Afficheur, buzzer	Compter le temps, comparer avec l'alarme
Machine à laver	Bouton programme, capteur de niveau d'eau, capteur de température	Moteur, électrovanne, résistance	Enchaîner les phases selon le programme choisi
Télécommande TV	Boutons (20+)	LED infrarouge	Encoder chaque touche en séquence binaire (protocole NEC)
Thermostat	Capteur température, boutons réglage	Relais chauffage/clim, écran LCD	Comparer température mesurée au seuil, activer/désactiver
ABS voiture	4 capteurs de vitesse de roue	4 électrovannes de frein	100 comparaisons/seconde par roue
Badge d'accès	Lecteur RFID, horloge	Relais porte, LEDs, buzzer	Vérifier identité ET horaire autorisé

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

2.1.4 Pourquoi les microcontrôleurs sont-ils partout ?

Les 5 qualités qui expliquent leur succès

1. Intégration
Tout sur une seule puce : pas besoin de 10 composants séparés

2. Compacité
Tient dans un boîtier de 3 cm. Parfait pour montre, capteur, drone

3. Sobriété
15 mW en marche, 0,1 µW en veille. Des mois sur pile

4. Coût
Un ATmega328P coûte environ **2 €**.
Un ATtiny85 : **0.50 €**

5. Vitesse
16 millions d'opérations par seconde. Largement suffisant pour la plupart des applications

Chiffre clé : combien de µC dans le monde ?

On estime qu'il se vend environ **30 milliards** de microcontrôleurs par an dans le monde (source : IC Insights). C'est environ **4 µC par être humain par an**.
En comparaison, il se vend environ 400 millions de PC et 1.4 milliard de smartphones. Les microcontrôleurs sont donc **20 fois plus nombreux** que les smartphones.

Ce qu'il faut retenir

- Un microcontrôleur est un **ordinateur miniature complet sur une seule puce**.
- Il est **invisible** dans les objets mais **indispensable** à leur fonctionnement.
- Il suit toujours le même schéma : **lire** → **calculer** → **décider** → **agir**.
- Il est petit, pas cher, rapide et économe.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

2.2 Petite histoire des microcontrôleurs

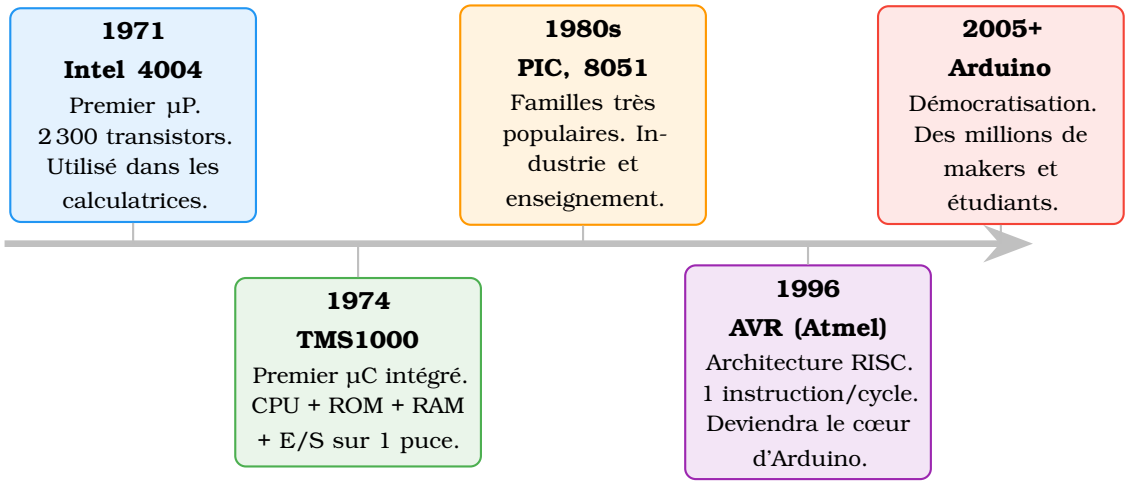
L'évolution des µC, comme celle des téléphones

Imaginez l'évolution des téléphones :

- **1970s** : téléphone fixe à fil (= premier microprocesseur Intel 4004, énorme et limité)
- **1990s** : premier portable Nokia (= premiers µC compacts comme le PIC et l'AVR)
- **2020s** : smartphone (= µC modernes ARM Cortex-M avec WiFi, Bluetooth, capteurs intégrés)

Comme les téléphones, les µC sont devenus plus **petits**, plus **puissants** et beaucoup moins **chers**, tout en faisant **plus de choses**.

Les grandes dates



En résumé : en 50 ans, on est passé de puces à 2 300 transistors utilisées dans les calculatrices à des systèmes de plusieurs milliards de transistors capables de piloter une voiture autonome. Et le mouvement continue.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

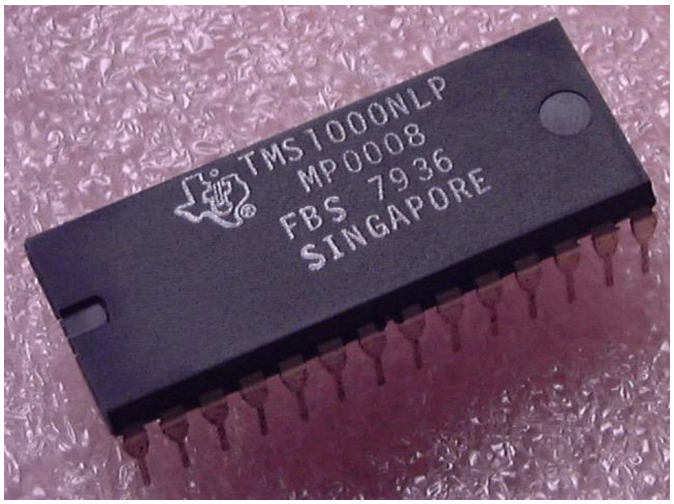


Figure 2.3 – Le TMS1000 (1974) : l'ancêtre de tous les microcontrôleurs modernes.

2.3 Microcontrôleur vs microprocesseur : quelle différence ?

C'est l'une des questions les plus fréquentes des débutants. Prenons le temps de bien l'expliquer.

Le restaurant vs le food truck

Microprocesseur = Restaurant complet

- Le chef (CPU) est excellent...
- ...mais il lui faut une cuisine (mémoire externe),
- des serveurs (circuits d'E/S),
- un bâtiment (carte mère),
- une équipe complète.
- **Puissant mais complexe et coûteux.**

Microcontrôleur = Food truck

- Le cuisinier (CPU) fait tout lui-même...
- ...dans un petit espace (une seule puce),
- avec sa propre plaque (E/S intégrées),
- son propre frigo (mémoire intégrée),
- tout est autonome.
- **Compact, autonome et économique.**

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

2.3.1 Comparaison détaillée

Table 2.2 – Microprocesseur vs microcontrôleur : comparaison point par point.

Critère	**Microprocesseur (µP)**	**Microcontrôleur (µC)**
Analogie	Restaurant avec équipe complète	Food truck autonome
Rôle	Calcul généraliste puissant	Contrôle dédié d'un système
Mémoire	Externe (Go de RAM, To de SSD)	Intégrée (Ko à Mo)
E/S	Via chipset et cartes additionnelles	GPIO directement sur la puce
Consommation	5 à 150 W (nécessite ventilateur)	1 µW à 500 mW (fonctionne sur pile)
Prix	10 € à 1 000+ €	0.20 € à 10 €
Exemples	Intel Core i7, AMD Ryzen 9, Apple M3	ATmega328P, STM32, ESP32, PIC
On l'utilise dans. . .	PC, serveur, smartphone, console de jeu	Réveil, thermostat, voiture, drone, Arduino

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

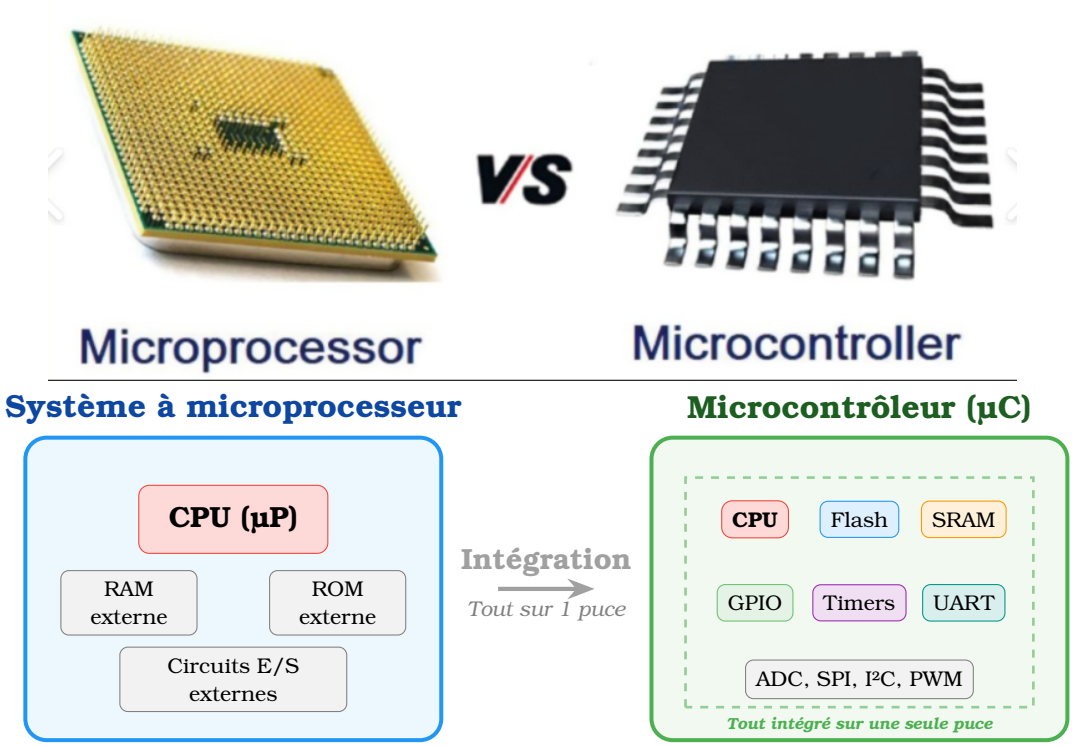


Figure 2.4 – Comparaison visuelle : le µP a besoin de composants externes, le µC intègre tout sur une puce.

Erreur courante

« **Un microcontrôleur, c'est un petit microprocesseur** » — FAUX !
Ce n'est pas une question de taille ou de puissance. La différence fondamentale est **l'intégration** :

- Le µP **ne contient que le CPU**. Il a besoin de composants externes.
- Le µC **contient tout** : CPU + mémoire + E/S + périphériques, sur une seule puce.

Un µP très petit (comme un ARM Cortex-A7) reste un µP. Un µC très puissant (comme un ESP32 avec WiFi et Bluetooth) reste un µC.
Ce qui les différencie, c'est le niveau d'intégration, pas la puissance.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

</> Pourquoi un thermostat n'utilise pas un Intel Core i7 ?

Un thermostat doit :

- lire une température (1 valeur) ;
- la comparer à un seuil (1 soustraction) ;
- allumer ou éteindre le chauffage (1 sortie).

Total des besoins : quelques centaines d'instructions par seconde, quelques octets de mémoire.
Un Intel Core i7 peut exécuter **100 milliards** d'instructions par seconde. C'est comme utiliser un **Boeing 747 pour aller chercher le pain**.
Le µC ATmega328P fait le travail parfaitement à 16 MHz, coûte **2 €** au lieu de **400 €**, consomme **15 mW** au lieu de **125 W**, et tient dans un boîtier de **3 cm** au lieu d'un système de refroidissement de 30 cm.

2.4 Les microcontrôleurs dans le monde réel

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Où trouve-t-on des microcontrôleurs ? Partout !

	Domaine	Exemples concrets	Nb µC
	Maison	Four, thermostat, lave-linge, cafetière, alarme, éclairage LED, volets roulants	10–30
	Automobile	Gestion moteur, ABS, airbags, climatisation, tableau de bord, capteurs de recul, fermeture centralisée	50–100
	Santé	Thermomètre frontal, tensiomètre, pompe à insuline, oxymètre, pacemaker	1–5
	Industrie	Robot soudeur, capteur de vibration, automate de ligne de production	10–50
	IoT	Station météo connectée, serrure connectée, tracker GPS pour animaux, arrosage automatique	1–3
	Loisirs	Drone, robot aspirateur, console de jeu rétro, guirlande LED programmable	3–20

La voiture : un réseau de microcontrôleurs

Une voiture moderne contient typiquement **50 à 100 microcontrôleurs** qui communiquent entre eux via un réseau appelé **CAN bus** (*Controller Area Network*).
Chaque µC est dédié à une fonction :

- Gestion moteur
- Boîte de vitesses
- ABS / ESP
- Airbags
- Direction assistée
- Climatisation
- Tableau de bord
- Éclairage
- Fermeture centralisée
- Lève-vitres
- Rétroviseurs
- Essuie-glaces auto

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Ils fonctionnent tous en parallèle, chacun exécutant son propre programme de manière indépendante.

2.5 La famille AVR : notre terrain de jeu

2.5.1 Pourquoi apprendre avec les AVR ?

La famille AVR : idéale pour débuter

Les microcontrôleurs **AVR** ont été créés en 1996 par deux étudiants norvégiens, Alf-Egil Bogen et Vegard Wollan, à l'Université de Trondheim. Leur entreprise **Atmel** a commercialisé cette architecture, rachetée en 2016 par **Microchip Technology**.

Pourquoi sont-ils parfaits pour apprendre ?

Lisibilité
Le jeu d'instructions est petit (131 instructions), régulier et bien documenté. Chaque instruction fait **une seule chose**.

Écosystème
Arduino = des millions de tutoriels, forums, bibliothèques. Vous ne serez **jamais seul**.

Accessibilité
Une carte Arduino Uno coûte **10 €**. Simulation gratuite sur Tinkercad.

En pratique : dans ce cours, nous utiliserons l'**ATmega328P**, le µC qui équipe la carte Arduino Uno.

Pourquoi pas un ARM Cortex-M directement ?

C'est comme apprendre à conduire :

- On commence avec une **petite voiture** (= AVR) : peu de puissance, mais on comprend tout.
- Ensuite, on peut passer à une **voiture de sport** (= ARM) : plus puissante, mais les bases sont les mêmes.

Les concepts appris sur AVR (registres, mémoire, interruptions, E/S) s'ap-

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

pliquent à **toutes les architectures**.

Table 2.3 – Les deux sous-familles AVR que vous rencontrerez.

Famille	**Exemple**	**Flash**	**RAM**	**Broches**	**Usage typique**
ATtiny	ATtiny85	8 Ko	512 o	8	Gadget, LED, capteur simple
ATmega	**ATmega328**	**32 Ko**	**2 Ko**	**28**	**Arduino Uno, ce cours**

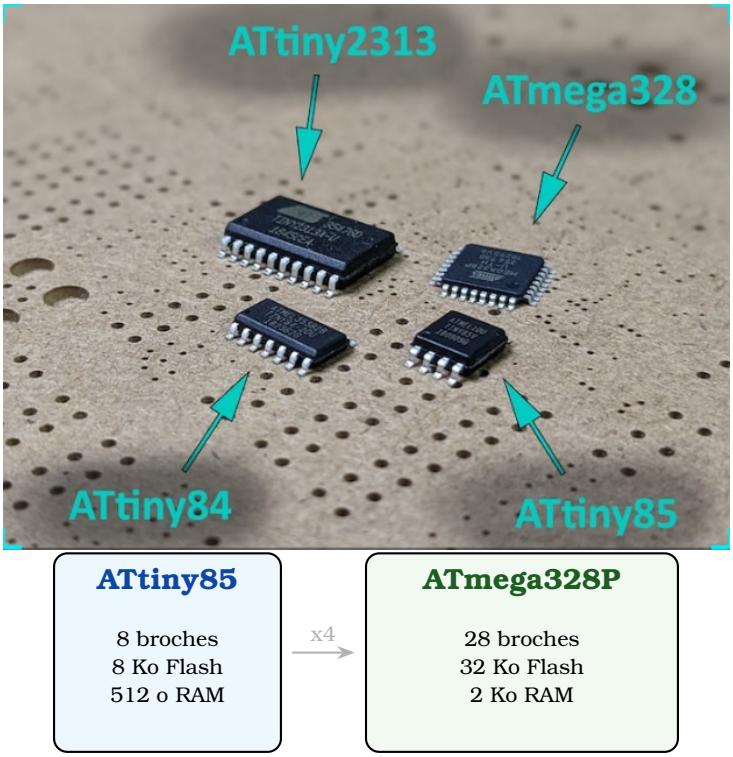


Figure 2.5 – Comparaison ATtiny85 vs ATmega328P.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

2.6 Architecture interne : visite guidée de l'ATmega328P

Nous allons maintenant « ouvrir le capot » du µC et examiner chaque composant, un par un.

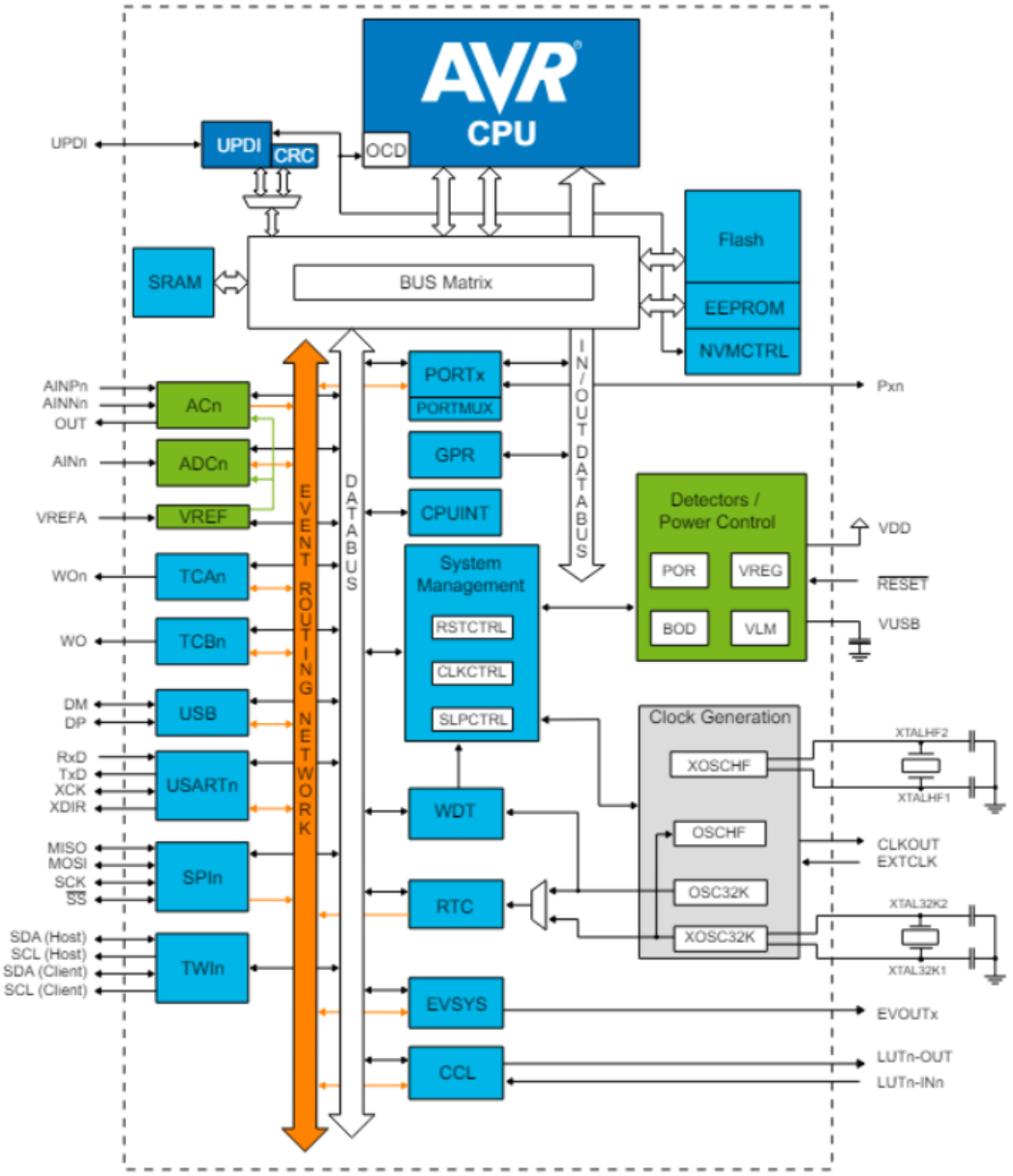


Figure 2.6 – Schéma de l'architecture d'un microcontrôleur AVR.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Le µC comme une petite usine

Imaginez une usine miniature :

- Le **directeur** (CPU) décide quoi faire.
- Le **calculateur** (ALU) effectue les opérations.
- Les **tiroirs sur le bureau** (registres) permettent de manipuler rapidement des valeurs.
- Le **manuel d'instructions** (Flash) contient la procédure à suivre.
- Le **bloc-notes** (SRAM) sert à noter des résultats temporaires.
- Le **coffre-fort** (EEPROM) conserve les données importantes même en cas de coupure de courant.
- Les **portes et fenêtres** (E/S) permettent de communiquer avec l'extérieur.
- L'**horloge murale** (oscillateur) rythme le travail.

2.6.1 Architecture RISC et Harvard : deux choix de conception

Architecture RISC (*Reduced Instruction Set Computing*)

L'architecture **RISC** repose sur un principe simple : **moins d'instructions, mais chacune est très rapide**.

Analogie : Imaginez deux cuisiniers.

- Le premier (CISC) connaît 500 recettes, mais chaque recette prend un temps variable (1 à 10 minutes).
- Le second (RISC) ne connaît que 130 recettes simples, mais chacune prend **exactement 1 minute**.

Le second est plus **prévisible** et souvent plus **rapide** dans l'ensemble, car le rythme est régulier.

L'AVR est RISC : il possède **131 instructions**, et la grande majorité s'exécute en **un seul cycle**.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Architecture Harvard

L'architecture **Harvard** utilise **deux mémoires séparées** avec **deux bus distincts** :

- un bus pour les **instructions** (programme en Flash) ;
- un bus pour les **données** (variables en SRAM).

Avantage : le processeur peut lire l'instruction suivante **en même temps** qu'il manipule les données de l'instruction en cours. C'est comme lire la recette de cuisine tout en mélangeant les ingrédients : on gagne du temps.

Par contraste : l'architecture *Von Neumann* (utilisée dans les PC) partage un seul bus pour les instructions et les données. C'est plus simple mais plus lent.

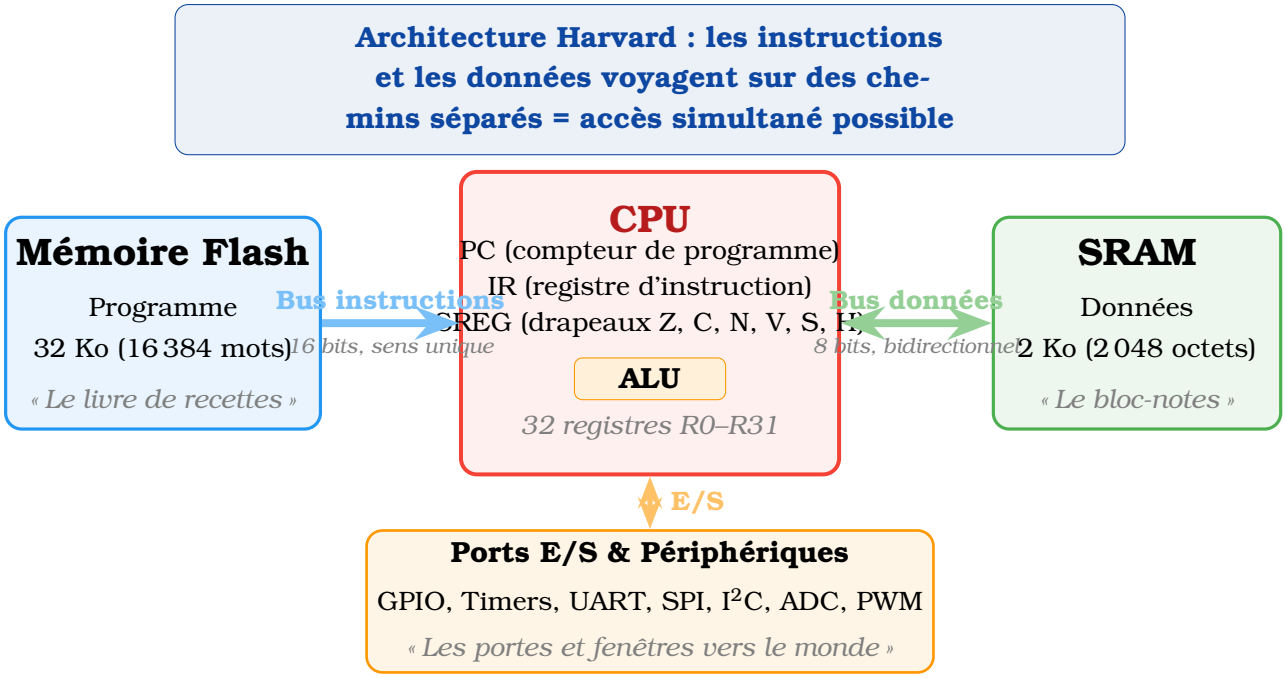


Figure 2.7 – Architecture Harvard de l'ATmega328P. Les deux bus séparés permettent de lire la prochaine instruction pendant que l'instruction en cours manipule des données.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

2.6.2 Le CPU et l'ALU : le cerveau et le calculateur

Le CPU (*Central Processing Unit*)

Le CPU est le **chef d'orchestre**. Il ne calcule pas lui-même (c'est le rôle de l'ALU), mais il :

1. **va chercher** l'instruction suivante en Flash (grâce au compteur de programme PC) ;
2. **la décode** : « c'est une addition ? une comparaison ? un saut ? » ;
3. **la fait exécuter** par l'ALU ou par un périphérique ;
4. **range le résultat** dans un registre ou en mémoire ;
5. **passe à l'instruction suivante** (PC += 1).

L'ALU (*Arithmetic and Logic Unit*)

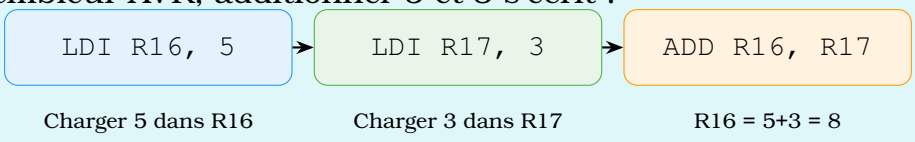
L'ALU est le **calculateur**. Elle sait faire exactement 6 types d'opérations :

1. **Additionner** : $5 + 3 = 8$
2. **Soustraire** : $10 - 4 = 6$
3. **Comparer** : $7 > 3$?
4. **ET / OU / XOR logique** : masquage de bits
5. **Décaler** : multiplier/diviser par 2
6. **Mettre à jour les drapeaux** : le résultat est-il zéro ? négatif ? y a-t-il une retenue ?

C'est tout ! Et pourtant, avec ces 6 opérations, on peut faire tout ce qu'un programme fait.

</> L'ALU en action : 5 + 3

En assembleur AVR, additionner 5 et 3 s'écrit :



Après l'exécution : R16 contient 8.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

2.6.3 Les 32 registres : les tiroirs ultra-rapides

Les registres, c'est comme les poches de votre veste

Quand vous faites des courses :

- Vos **poches** = les registres. Accès instantané, mais capacité limitée.
- Votre **sac à dos** = la SRAM. Plus de place, mais il faut l'ouvrir (plus lent).
- Votre **armoire à la maison** = la Flash. Beaucoup de place, mais il faut rentrer chez vous.

Le processeur essaie toujours de travailler avec ses « poches » (registres) d'abord, car c'est **instantané**.

Les AVR ont **32 registres de 8 bits**, numérotés R0 à R31. Chacun peut stocker un nombre entre 0 et 255.

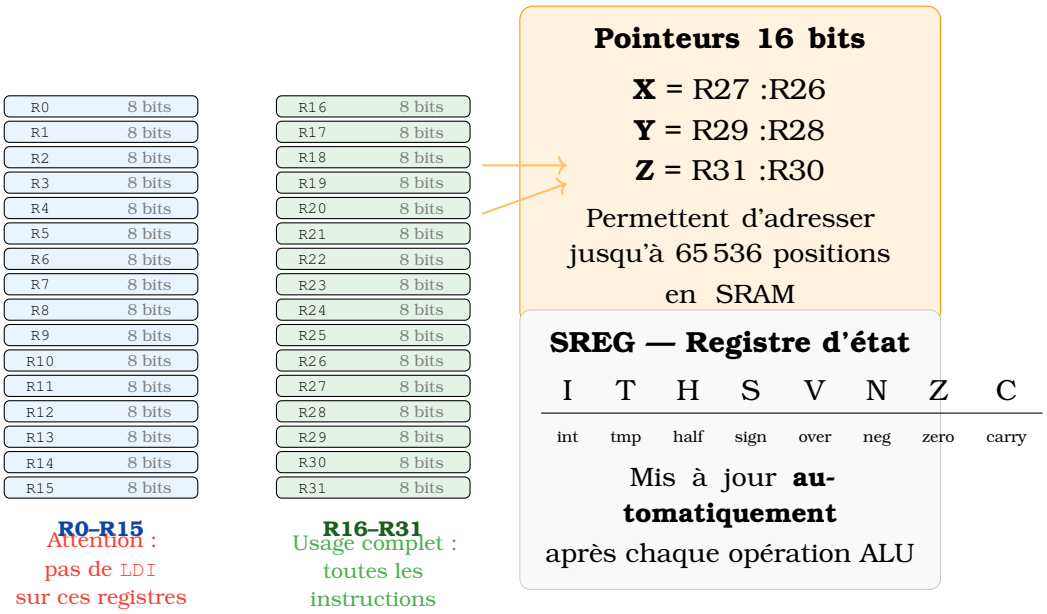


Figure 2.8 – Les 32 registres de l'AVR. R16–R31 sont les plus utilisés car ils acceptent toutes les instructions. R26–R31 peuvent former des pointeurs 16 bits (X, Y, Z).

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Pourquoi R16–R31 et pas R0–R15 ?

En assembleur AVR, l'instruction `LDI` (*Load Immediate* = charger une valeur directe) ne fonctionne **que sur R16 à R31**.
Cela signifie que pour écrire `LDI R16, 42` (mettre 42 dans R16), ça marche. Mais `LDI R5, 42` provoquera une **erreur d'assemblage**.
C'est une limitation de l'encodage des instructions sur 16 bits : il n'y a que 4 bits pour le numéro de registre dans le format de `LDI`, ce qui permet uniquement les registres 16 à 31.

2.6.4 Les trois mémoires, expliquées simplement

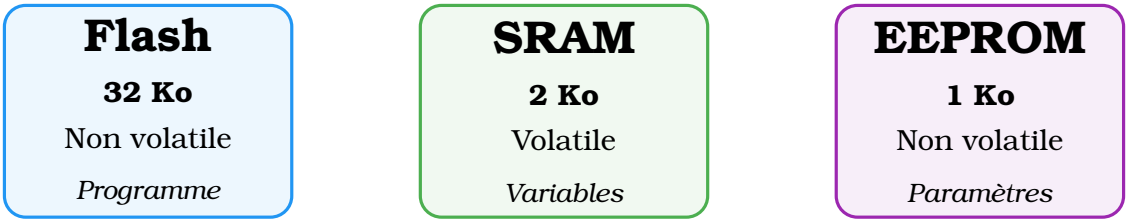


Figure 2.9 – Les trois mémoires de l'ATmega328P.

Table 2.4 – Comparaison détaillée des trois mémoires.

Mémoire	Taille	Persistante	Analogie	Contient
Flash	32 Ko	**Oui**	Livre de recettes	Le programme (vos instructions assembleur)
SRAM	2 Ko	**Non**	Bloc-notes	Variables, pile, résultats temporaires
EEPROM	1 Ko	**Oui**	Coffre-fort	Paramètres persistants (calibration, config)

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Erreur courante

« **32 Ko, c'est minuscule !** » — Pas pour un microcontrôleur !
32 Ko de Flash, c'est **16 384 instructions** assembleur (chaque instruction fait 2 octets). Pour donner une idée, un programme qui gère un thermostat complet (lecture capteur, affichage LCD, communication série, gestion d'alarme) tient en **environ 2 000 instructions**.
La Flash de l'ATmega328P peut donc contenir **8 programmes de ce type** simultanément.

En comparaison, un PC a besoin de Go de RAM parce qu'il exécute un système d'exploitation, un navigateur, un éditeur, un antivirus. . . en même temps. Le µC n'exécute qu'un seul programme, beaucoup plus simple.

2.6.5 Les périphériques : les outils spéciaux du µC

En plus du CPU et de la mémoire, le µC contient des **périphériques intégrés** qui étendent ses capacités sans ajouter de composants externes.

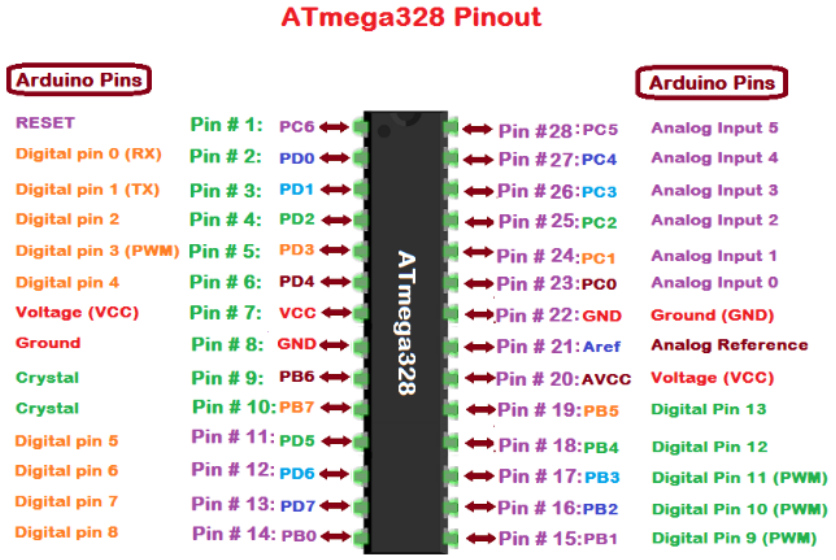


Figure 2.10 – Configuration des broches d'un microcontrôleurs AVR.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Les périphériques de l'ATmega328P

	Périphérique	Analogie	Exemple d'utilisation
	Timers(0, 1, 2)	Un chronomètre intégré	Faire clignoter une LED toutes les 500 ms, mesurer la durée d'un appui bouton
	UART	Un téléphone : envoyer / recevoir des messages	Afficher des données sur un terminal PC, communiquer avec un module Bluetooth
	SPI	Un talkie-walkie rapide	Piloter un écran LCD, lire une carte SD
	I²C	Un bus scolaire : un seul chemin, plusieurs passagers	Connecter 10 capteurs différents avec seulement 2 fils
	ADC(10 bits)	Un thermomètre numérique intégré	Lire un potentiomètre, mesurer la tension d'une batterie, lire un capteur de lumière
	PWM	Un variateur de lumière	Faire varier la luminosité d'une LED, contrôler la vitesse d'un moteur

L'ADC en action : mesurer une température

Supposons que vous connectez un capteur de température (type LM35) à la broche A0 de l'Arduino :

1. Le capteur génère une **tension analogique** proportionnelle à la température : $V = \text{température} \times 10\,\text{mV}/°\text{C}$
2. L'ADC 10 bits du µC convertit cette tension en un **nombre** entre 0 et

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

1023 :

$$\text{valeur} = \frac{V \times 1023}{5\,\text{V}}$$

3. Le programme calcule la température :

$$T = \frac{\text{valeur} \times 5}{1023} \times 100 \quad \text{(en °C)}$$

4. Le résultat est affiché sur un écran ou envoyé par UART au PC.

Bilan : le µC a lu une grandeur physique (température), l'a convertie en nombre, a fait un calcul, et a communiqué le résultat. Tout cela en quelques microsecondes.

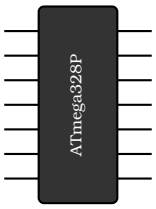


Figure 2.11 – L'ATmega328P en boîtier DIP-28.

2.7 L'horloge, le reset et le cycle d'instruction

2.7.1 L'horloge : le métronome du µC

L'horloge, c'est comme le métronome d'un musicien

Un musicien joue les notes au rythme du métronome. Plus le métronome bat vite, plus la musique avance rapidement.
Le µC fonctionne pareil :

- Chaque « tic » de l'horloge = un cycle.
- À chaque cycle, le µC avance d'un pas dans le programme.
- Plus l'horloge est rapide, plus le µC travaille vite.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Horloge système

Le µC est cadencé par un **oscillateur à quartz** qui produit un signal régulier.
L'ATmega328P sur Arduino Uno fonctionne à :

$$f = 16\,\text{MHz} = 16\,000\,000 \text{ cycles par seconde}$$

Chaque cycle dure :

$$T = \frac{1}{f} = \frac{1}{16\,000\,000} = 62{,}5\,\text{ns} = 0{,}0000000625 \text{ seconde}$$

En d'autres termes : le µC peut exécuter une instruction en 62,5 ns, soit **16 millions d'instructions par seconde**.

Pour donner une idée de la vitesse. . .

En **1 seconde**, l'ATmega328P exécute autant d'instructions qu'un humain qui ferait une opération toutes les secondes pendant. . .

$$\frac{16\,000\,000}{3\,600 \times 24 \times 365} \approx \textbf{507 années}$$

Le µC fait en 1 seconde ce qu'un humain mettrait **5 siècles** à faire !

2.7.2 Le reset : redémarrer proprement

Reset

Le **reset** est l'équivalent d'un « redémarrage » du µC. Il :

- remet tous les registres à zéro ;
- replace le compteur de programme (PC) à l'adresse `0x0000` (= première instruction) ;
- redémarre le programme depuis le début.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

1. **Mise sous tension** (*Power-on Reset*) : quand vous branchez l'alimentation ou connectez l'USB. *Comme allumer votre PC.*
2. **Bouton reset** (*External Reset*) : le bouton RESET de la carte Arduino. *Comme le bouton reset d'une console de jeu.*
3. **Chien de garde** (*Watchdog Reset*) : un timer spécial vérifie que le programme ne s'est pas « planté ». Si le programme ne « caresse le chien » régulièrement, le µC se redémarre automatiquement. *Comme un vigile qui vérifie que l'employé est toujours éveillé.*
4. **Tension trop basse** (*Brown-out Reset*) : si la batterie est presque vide, le µC se réinitialise plutôt que de fonctionner de manière imprévisible. *Comme un ordinateur qui s'éteint quand la batterie est à 1%.*

2.7.3 Le cycle d'instruction : comment le µC exécute votre programme

Le cycle d'instruction, c'est comme suivre une recette de cuisine

Imaginez que vous cuisinez en suivant un livre de recettes :

1. **FETCH** (Chercher) : vous lisez l'instruction suivante dans le livre. « *Ajouter 200g de farine.* »
2. **DECODE** (Comprendre) : vous comprenez ce qu'il faut faire. « *OK, je dois prendre de la farine et en mettre 200g dans le saladier.* »
3. **EXECUTE** (Exécuter) : vous faites l'action. Vous prenez la farine et la versez.
4. **WRITE-BACK** (Ranger) : vous posez le résultat. La farine est dans le saladier, prête.
5. **NEXT** : vous passez à la ligne suivante du livre et recommencez.

Le µC fait exactement pareil, mais **16 millions de fois par seconde**.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

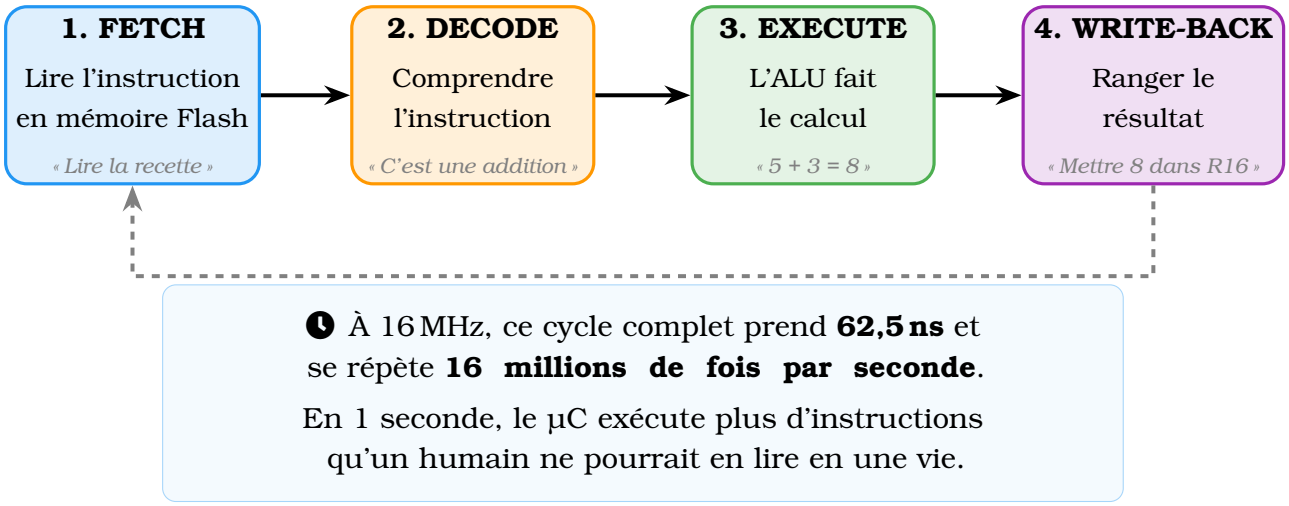


Figure 2.12 – Le cycle Fetch-Decode-Execute-Writeback, avec analogies culinaires pour chaque étape.

</> Exemple pas-à-pas : allumer une LED

Supposons que votre programme contient l'instruction `SBI PORTB, 5` (allumer la LED sur la broche 13 de l'Arduino, qui correspond au bit 5 du port B) :

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Étape	Ce que fait le µC	Explication pour un débutant
1. Fetch	Lit `0x9A2D` en Flash	Le µC va chercher la prochaine instruction dans son « livre de recettes » (la Flash). Il trouve le code binaire `0x9A2D`.
2. Decode	Reconnaît `SBI PORTB, 5`	Le décodeur traduit : « c'est l'instruction SBI, qui veut dire : mettre le bit numéro 5 du registre PORTB à 1 ».
3. Execute	Met le bit 5 de PORTB à 1	L'opération est effectuée : le bit passe de 0 à 1. Électriquement, la broche PB5 passe de 0 V à 5 V.
4. Write-back	LED allumée !	Le courant traverse la LED (via une résistance), et elle s'allume.
5. PC += 1	Passe à l'instruction suivante	Le compteur de programme avance d'un cran. Le µC est prêt pour l'instruction d'après.

Durée totale : 125 ns (2 cycles à 16 MHz). Votre œil ne verra rien : la LED s'allume « instantanément ».

2.8 Compilation croisée : du PC au µC

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

2.8.1 Pourquoi ne peut-on pas programmer directement le µC ?

Écrire un livre dans une langue étrangère

Imaginez que vous devez écrire des instructions pour un ouvrier qui ne parle que le **chinois**.

- Vous, vous parlez **français** (= langage C).
- L'ouvrier ne comprend que le **chinois** (= code machine AVR, des suites de 0 et de 1).
- Vous avez besoin d'un **traducteur** sur votre bureau (= le compilateur `avr-gcc` sur votre PC).
- Une fois traduit, vous envoyez le texte à l'ouvrier (= téléversement avec `avrdude`).

C'est exactement ce qu'est la **compilation croisée** : traduire sur votre PC un programme en C vers le langage que comprend le µC AVR.

Compilation croisée (*Cross-compilation*)

La **compilation croisée** consiste à compiler un programme sur une machine **hôte** (votre PC, architecture x86 ou x64) afin de produire un fichier exécutable destiné à une architecture **cible** totalement différente (le µC AVR, architecture 8 bits).

C'est nécessaire car :

- le µC n'a pas de clavier, pas d'écran, pas de système d'exploitation pour faire tourner un compilateur ;
- ses 2 Ko de RAM ne suffiraient pas pour un compilateur ;
- le format des instructions AVR (16 bits) est incompatible avec celui du PC (64 bits).

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

2.8.2 La chaîne de compilation, étape par étape

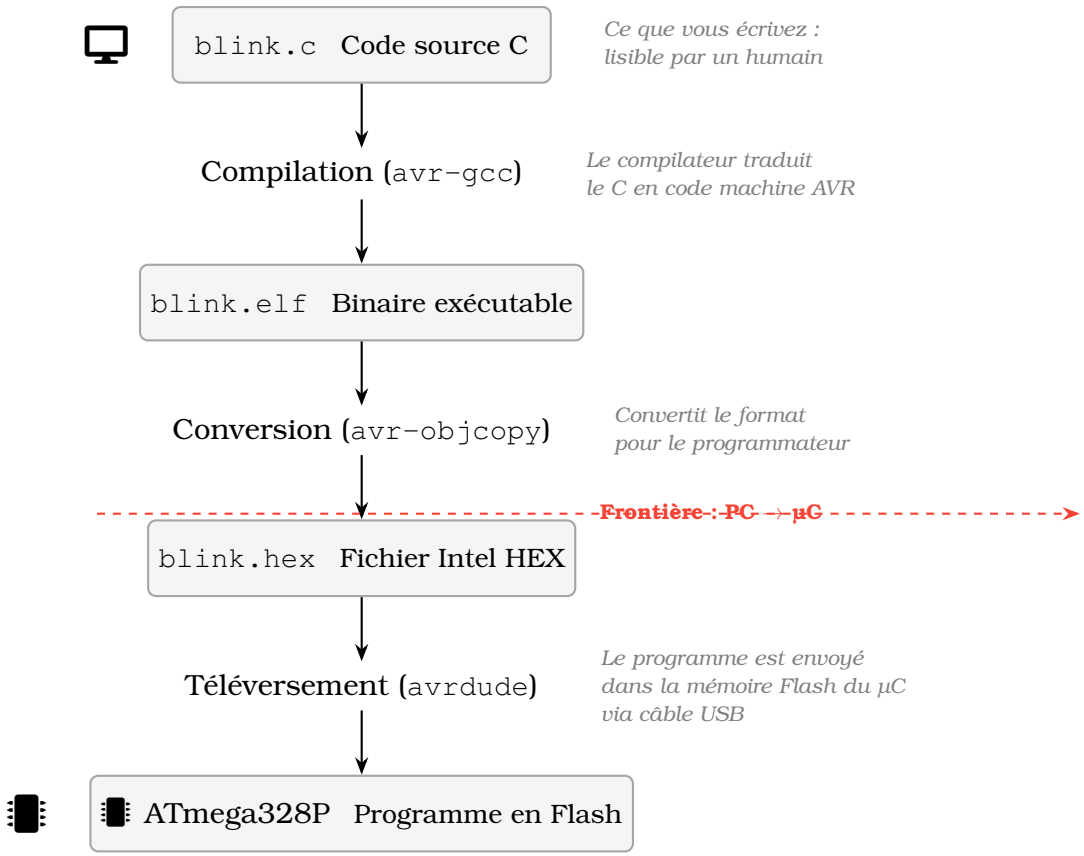


Figure 2.13 – Chaîne simplifiée de compilation croisée pour AVR. Vous écrivez en C sur votre PC, le compilateur traduit, et le résultat est envoyé dans le µC.

Les 3 commandes essentielles

En pratique, tout tient en 3 commandes dans un terminal :

1. **Compiler** (traduire le C en code machine AVR) :

```
avr-gcc -mmcu=atmega328p -DF_CPU=16000000UL -Os \
    -o blink.elf blink.c
```

`-mmcu` = quel µC, `-DF_CPU` = fréquence d'horloge, `-Os` = optimiser pour la taille

2. **Convertir** (format compréhensible par le programmateur) :

```
avr-objcopy -O ihex -R .eeprom blink.elf blink.hex
```

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

3. Téléverser (envoyer dans la Flash du µC) :

```
avrdude -c arduino -p atmega328p -P /dev/ttyUSB0 \
    -U flash:w:blink.hex:i
```

`-P /dev/ttyUSB0` = le port USB où est connecté l'Arduino (sous Windows : `COM3`)

Note : L'IDE Arduino fait tout cela automatiquement quand vous cliquez sur « Téléverser ». Mais il est important de comprendre ce qui se passe en coulisses.

2.9 Votre premier programme : faire clignoter une LED

2.9.1 Le « Hello World » de l'embarqué

Dans le monde de la programmation PC, le premier programme est traditionnellement « Hello World » : afficher un texte à l'écran.

Dans le monde de l'embarqué, il n'y a pas d'écran. Le premier programme est donc **faire clignoter une LED**. C'est le test le plus simple pour vérifier que toute la chaîne fonctionne.

Listing 2.1 – Le « Blink » : votre premier programme Arduino. Chaque ligne est commentée pour un débutant.

```
// setup() s'execute UNE SEULE FOIS au demarrage
void setup() {
    // Configurer la broche 13 comme SORTIE
    // (broche 13 = LED integree sur Arduino Uno)
    pinMode(13, OUTPUT);
}

// loop() s'execute EN BOUCLE, indefiniment
void loop() {
    // Mettre la broche 13 a l'etat HAUT (5V)
    // = la LED s'allume
    digitalWrite(13, HIGH);

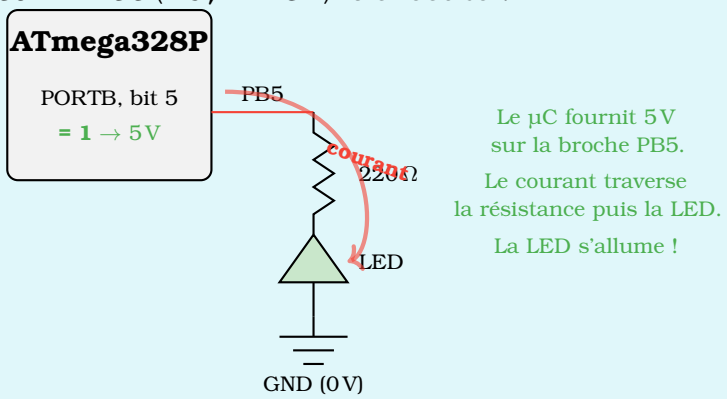
    // Attendre 1000 millisecondes = 1 seconde
```

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

```
15      delay(1000);
16
17      // Mettre la broche 13 a l'etat BAS (0V)
18      // = la LED s'eteint
19      digitalWrite(13, LOW);
20
21      // Attendre encore 1 seconde
22      delay(1000);
23
24      // Le programme revient automatiquement au debut
25      // de loop() : la LED reclignote.
26  }
```

</> Ce qui se passe physiquement

Quand `digitalWrite(13, HIGH)` s'exécute :



La résistance de 220 Ω limite le courant à environ 20 mA pour ne pas griller la LED.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

2.9.2 Outils de développement

Table 2.5 – Les outils pour commencer — du plus simple au plus avancé.

	Outil	Pour qui ?	Coût	Matériel ?
☁	**Tinkercad**	**Débutant complet.** Simulation en ligne, rien à installer.	Gratuit	Non
💻	**Arduino IDE**	Débutant avec une carte Arduino. Simple et intuitif.	Gratuit	Oui
⚙	**Microchip Studio**	Intermédiaire. Débogage pas-à-pas, vue des registres en temps réel.	Gratuit	Optionnel
🧪	**Proteus**	Simulation avancée de circuits complets avec visualisation oscilloscope.	Payant	Non

Conseil pour démarrer

Si vous n'avez pas de matériel :

1. Commencez avec **Tinkercad Circuits** (https://www.tinkercad.com/circuits) — simulation gratuite dans le navigateur.
2. Créez un circuit avec un Arduino Uno et une LED.
3. Copiez le programme Blink ci-dessus.
4. Cliquez sur « Démarrer la simulation ».
5. Observez la LED clignoter !

C'est exactement ce que vous ferez en TP.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

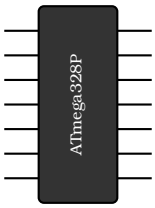


Figure 2.14 – L'ATmega328P en boîtier DIP-28.

2.10 Conclusion du chapitre

Ce que vous avez appris

Ce chapitre vous a fait découvrir le microcontrôleur « de l'extérieur vers l'intérieur » :

1. **Ce que c'est :** un ordinateur miniature complet sur une seule puce (CPU + mémoire + E/S + périphériques).
2. **Où on le trouve :** partout ! Réveil, cafetière, voiture, badge, thermostat, drone. . .
3. **La différence avec un µP :** le µC intègre tout, le µP a besoin de composants externes. *Food truck vs restaurant.*
4. **La famille AVR :** architecture RISC, 131 instructions, idéale pour apprendre. L'ATmega328P est notre µC de référence.
5. **L'architecture interne :**
   - CPU (le chef) + ALU (le calculateur)
   - 32 registres (les tiroirs ultra-rapides)
   - Flash (le programme), SRAM (les données), EEPROM (les paramètres)
   - Périphériques (timers, UART, ADC, PWM. . .)
6. **Le cycle d'exécution :** Fetch → Decode → Execute → Write-back, 16 millions de fois par seconde.
7. **La compilation croisée :** on écrit sur PC, on compile pour AVR, on

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

téléverse dans le µC.

Fiche de révision express

Concept	**Ce qu'il faut retenir**
Microcontrôleur	Ordinateur complet sur 1 puce : CPU + mémoire + E/S
µP vs µC	µP = cerveau seul (restaurant). µC = tout intégré (food truck).
Architecture	Harvard (2 bus séparés) + RISC (instructions simples et rapides)
Registres	32 tiroirs R0–R31, 8 bits chacun. R16–R31 = usage complet.
Flash	32 Ko, non volatile, contient le programme.
SRAM	2 Ko, volatile, contient les variables.
EEPROM	1 Ko, non volatile, contient les paramètres.
Horloge	16 MHz = 16 M instructions/s = 1 instruction en 62,5 ns
Cycle d'instruction	Fetch → Decode → Execute → Write-back
Compilation croisée	Écrire sur PC → compiler pour AVR → téléverser dans le µC

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Fin du Chapitre 2 — Introduction aux microcontrôleurs AVR

Vous connaissez maintenant la **machine** qui exécutera vos programmes assembleur.

Chapitre suivant : ***Fondements du langage assembleur AVR***

Vous allez écrire vos premières instructions : `LDI`, `MOV`, `ADD`...
et voir concrètement comment le µC les exécute, registre par registre.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Table des matières détaillée